



Unit-2

Digital Image Processing (Visvesvaraya Technological University)



Scan to open on Studocu

IMAGE ENHANCEMENT

Basic Gray Level Transformations:

The study of image enhancement techniques is done by discussing gray-level transformation functions. These are among the simplest of all image enhancement techniques. The values of pixels, before and after processing, will be denoted by r and s , respectively. As indicated in the previous section, these values are related by an expression of the form $s=T(r)$, where T is a transformation that maps a pixel value r into a pixel value s . Since we are dealing with digital quantities, values of the transformation function typically are stored in a one-dimensional array and the mappings from r to s are implemented via table lookups. For an 8-bit environment, a lookup table containing the values of T will have 256 entries. As an introduction to gray-level transformations, consider Fig. 1.1, which shows three basic types of functions used frequently for image enhancement: linear (negative and identity transformations), logarithmic (log and inverse-log transformations), and power-law (n th power and n th root transformations). The identity function is the trivial case in which output intensities are identical to input intensities. It is included in the graph only for completeness.

Image Negatives:

The negative of an image with gray levels in the range $[0, L-1]$ is obtained by using the negative transformation shown in Fig.1.1, which is given by the expression

$$s = L - 1 - r.$$

Reversing the intensity levels of an image in this manner produces the equivalent of a photographic negative. This type of processing is particularly suited for enhancing white or gray detail embedded in dark regions of an image, especially when the black areas are dominant in size.

Digital Image Processing

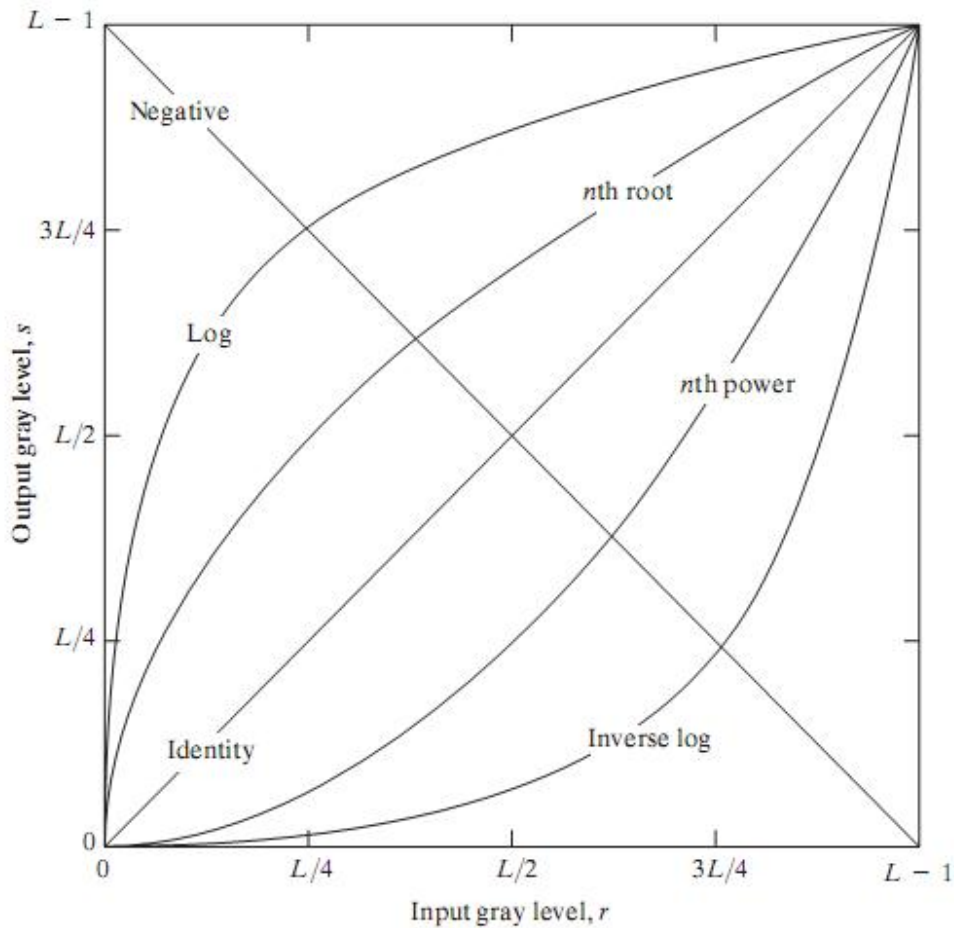


Fig.1.1 Some basic gray-level transformation functions used for image enhancement

Log Transformations:

The general form of the log transformation shown in Fig.1.1 is

$$s = c \log(1 + r)$$

where c is a constant, and it is assumed that $r \geq 0$. The shape of the log curve in Fig. 1.1 shows that this transformation maps a narrow range of low gray-level values in the input image into a wider range of output levels. The opposite is true of higher values of input levels. We would use a transformation of this type to expand the values of dark pixels in an image while compressing the higher-level values. The opposite is true of the inverse log transformation.

Digital Image Processing

Any curve having the general shape of the log functions shown in Fig. 1.1 would accomplish this spreading/compressing of gray levels in an image. In fact, the power-law transformations discussed in the next section are much more versatile for this purpose than the log transformation. However, the log function has the important characteristic that it compresses the dynamic range of images with large variations in pixel values. A classic illustration of an application in which pixel values have a large dynamic range is the Fourier spectrum. At the moment, we are concerned only with the image characteristics of spectra. It is not unusual to encounter spectrum values that range from 0 to or higher. While processing numbers such as these presents no problems for a computer, image display systems generally will not be able to reproduce faithfully such a wide range of intensity values. The net effect is that a significant degree of detail will be lost in the display of a typical Fourier spectrum.

Power-Law Transformations:

Power-law transformations have the basic form

$$s = cr^\gamma$$

where c and γ are positive constants. Sometimes Eq. is written as $s = c(r + \epsilon)^\gamma$

to account for an offset (that is, a measurable output when the input is zero). However, offsets typically are an issue of display calibration and as a result they are normally ignored in Eq. Plots of s versus r for various values of γ are shown in Fig. 1.2. As in the case of the log transformation, power-law curves with fractional values of γ map a narrow range of dark input values into a wider range of output values, with the opposite being true for higher values of input levels. Unlike the log function, however, we notice here a family of possible transformation curves obtained simply by varying γ . As expected, we see in Fig. 1.2 that curves generated with values of $\gamma > 1$ have exactly the opposite effect as those generated with values of $\gamma < 1$. Finally, we note that Eq. reduces to the identity transformation when $c = \gamma = 1$. A variety of devices used for image capture, printing, and display respond according to a power law. By convention, the exponent in the power-law equation is referred to as gamma. The process used to correct this power-law response phenomena is called gamma correction. For example, cathode ray tube (CRT) devices have an intensity-to-voltage response that is a power function, with exponents varying from approximately 1.8 to 2.5. With reference to the curve for $\gamma = 2.5$ in Fig. 1.2, we see that such display systems would tend to produce images that are darker than intended.

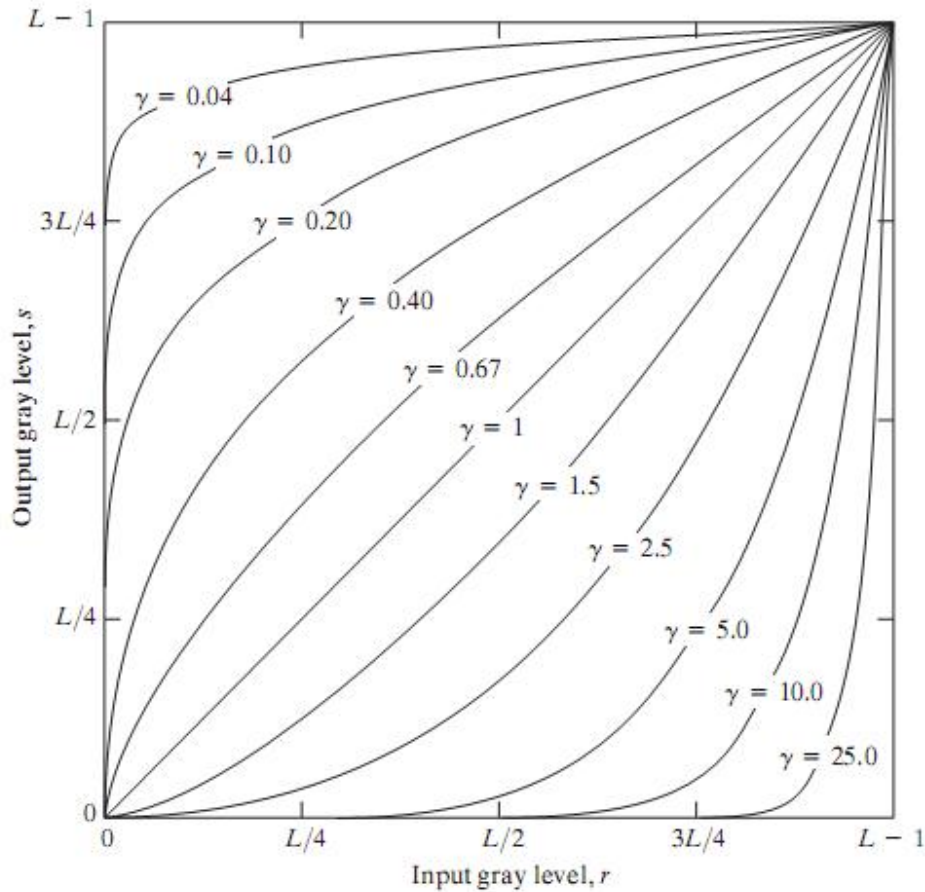


Fig.1.2 Plots of the equation $s = cr^\gamma$ for various values of γ ($c=1$ in all cases).

Piecewise-Linear Transformation Functions:

The principal advantage of piecewise linear functions over the types of functions we have discussed above is that the form of piecewise functions can be arbitrarily complex. In fact, as we will see shortly, a practical implementation of some important transformations can be formulated only as piecewise functions. The principal disadvantage of piecewise functions is that their specification requires considerably more user input.

Digital Image Processing

Contrast stretching:

One of the simplest piecewise linear functions is a contrast-stretching transformation. Low-contrast images can result from poor illumination, lack of dynamic range in the imaging sensor, or even wrong setting of a lens aperture during image acquisition. The idea behind contrast stretching is to increase the dynamic range of the gray levels in the image being processed.

Figure 1.3 (a) shows a typical transformation used for contrast stretching.

The locations of points (r_1, s_1) and (r_2, s_2) control the shape of the transformation

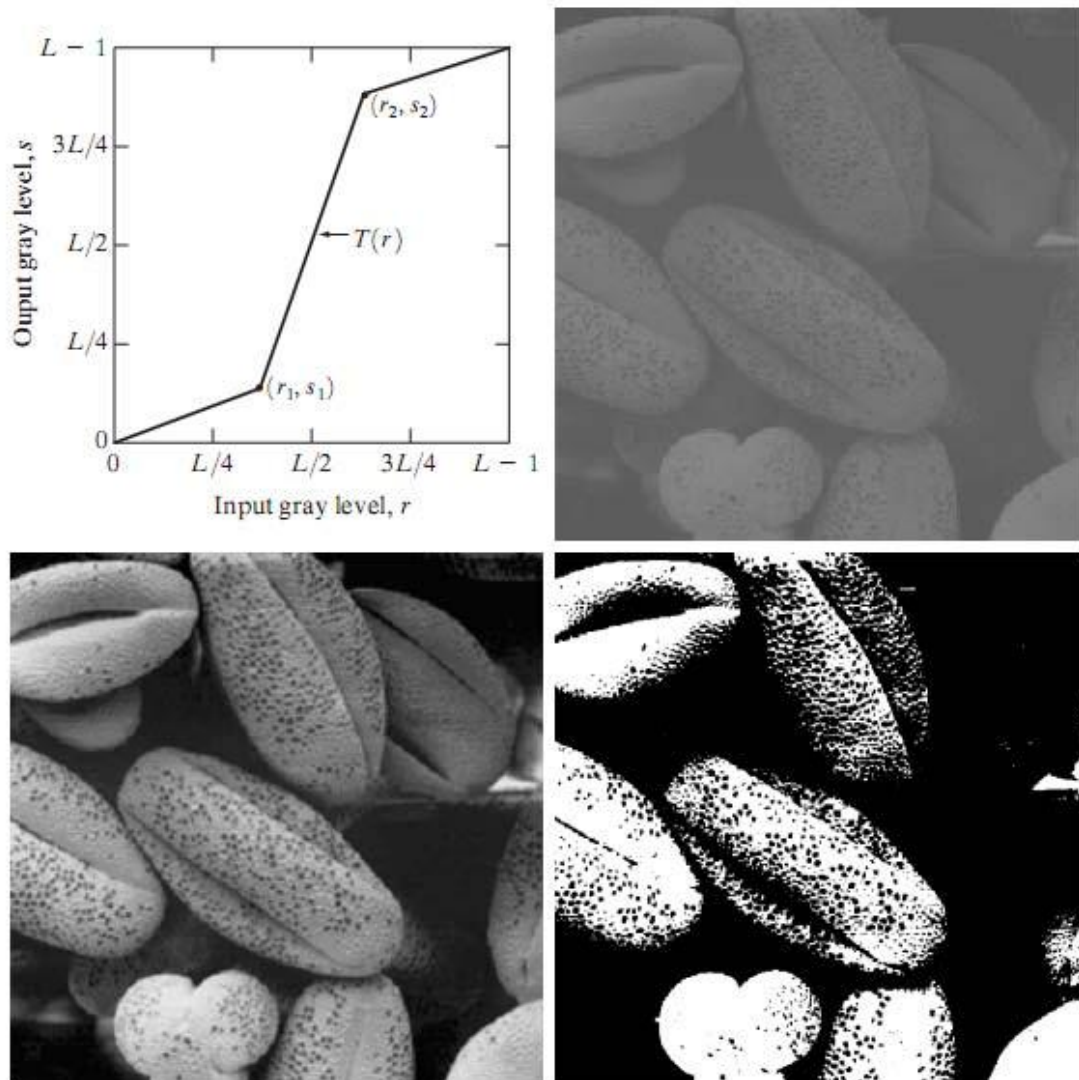


Fig.1.3 Contrast Stretching (a) Form of Transformation function (b) A low-contrast image (c) Result of contrast stretching (d) Result of thresholding.

function. If $r_1=s_1$ and $r_2=s_2$, the transformation is a linear function that produces no changes in gray levels. If $r_1=r_2, s_1=0$ and $s_2=L-1$, the transformation becomes a thresholding function that creates a binary image, as illustrated in Fig. 1.3 (b). Intermediate values of (r_1, s_1) and (r_2, s_2) produce various degrees of spread in the gray levels of the output image, thus affecting its contrast. In general, $r_1 \leq r_2$ and $s_1 \leq s_2$ is assumed so that the function is single valued and monotonically increasing. This condition preserves the order of gray levels, thus preventing the creation of intensity artifacts in the processed image.

Figure 1.3 (b) shows an 8-bit image with low contrast. Fig. 1.3(c) shows the result of contrast stretching, obtained by setting $(r_1, s_1) = (r_{\min}, 0)$ and $(r_2, s_2) = (r_{\max}, L-1)$ where $r_{\min}$ and $r_{\max}$ denote the minimum and maximum gray levels in the image, respectively. Thus, the transformation function stretched the levels linearly from their original range to the full range $[0, L-1]$. Finally, Fig. 1.3 (d) shows the result of using the thresholding function defined previously, with $r_1 = r_2 = m$, the mean gray level in the image. The original image on which these results are based is a scanning electron microscope image of pollen, magnified approximately 700 times.

Gray-level slicing:

Highlighting a specific range of gray levels in an image often is desired. Applications include enhancing features such as masses of water in satellite imagery and enhancing flaws in X-ray images. There are several ways of doing level slicing, but most of them are variations of two basic themes. One approach is to display a high value for all gray levels in the range of interest and a low value for all other gray levels. This transformation, shown in Fig. 1.4 (a), produces a binary image. The second approach, based on the transformation shown in Fig. 1.4 (b), brightens the desired range of gray levels but preserves the background and gray-level tonalities in the image. Figure 1.4(c) shows a gray-scale image, and Fig. 1.4 (d) shows the result of using the transformation in Fig. 1.4 (a). Variations of the two transformations shown in Fig. 1.4 are easy to formulate.

Digital Image Processing

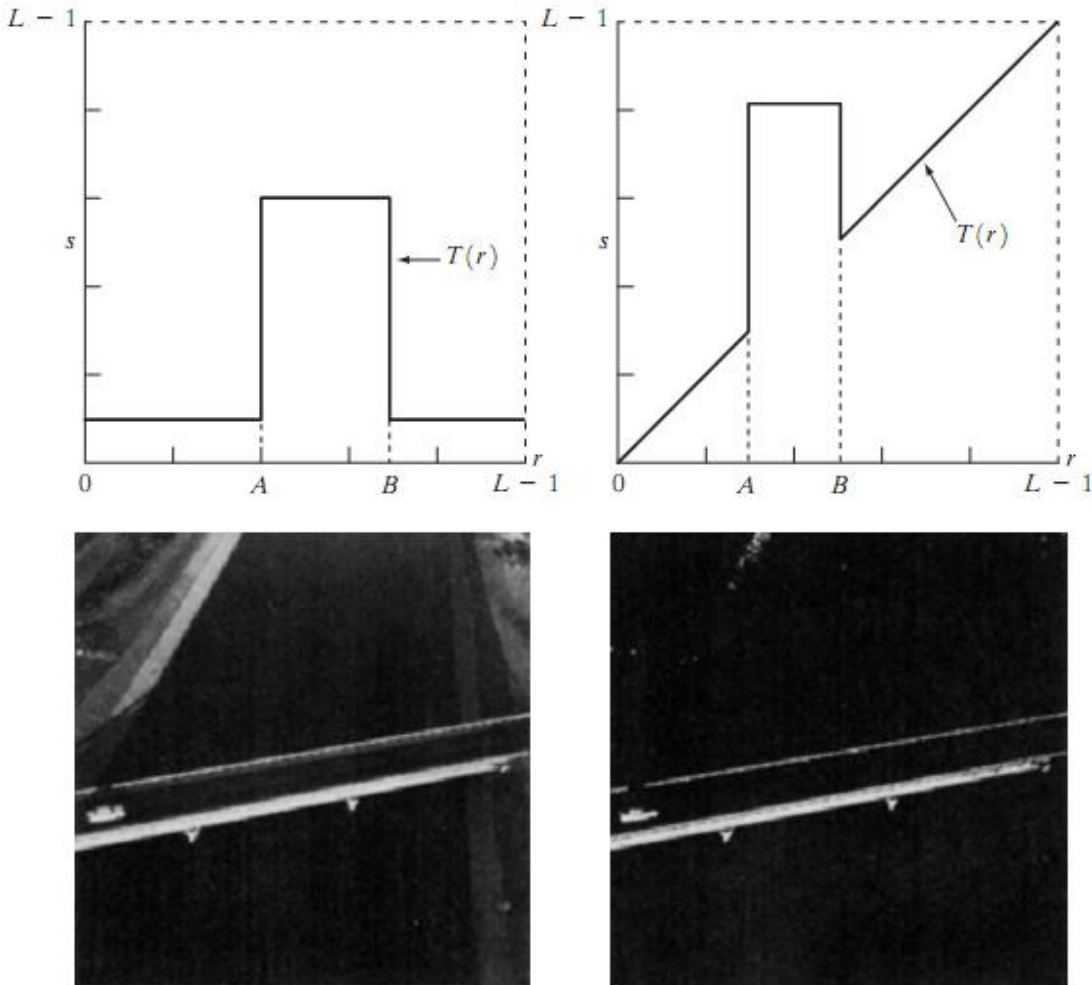


Fig.1.4 (a) This transformation highlights range $[A, B]$ of gray levels and reduce all others to a constant level (b) This transformation highlights range $[A, B]$ but preserves all other levels (c) An image (d) Result of using the transformation in (a).

Bit-plane slicing:

Instead of highlighting gray-level ranges, highlighting the contribution made to total image appearance by specific bits might be desired. Suppose that each pixel in an image is represented by 8 bits. Imagine that the image is composed of eight bit planes, ranging from bit-plane 0 for the least significant bit to bit plane 7 for the most significant bit. In terms of 8-bit bytes, plane 0

Digital Image Processing

contains all the lowest order bits in the bytes comprising the pixels in the image and plane 7 contains all the high-order bits. Figure 1.5 illustrates these ideas, and Fig. 1.7 shows the various bit planes for the image shown in Fig. 1.6. Note that the higher-order bits (especially the top four) contain the majority of the visually significant data. The other bit planes contribute to more subtle details in the image. Separating a digital image into its bit planes is useful for analyzing the relative importance played by each bit of the image, a process that aids in determining the adequacy of the number of bits used to quantize each pixel.

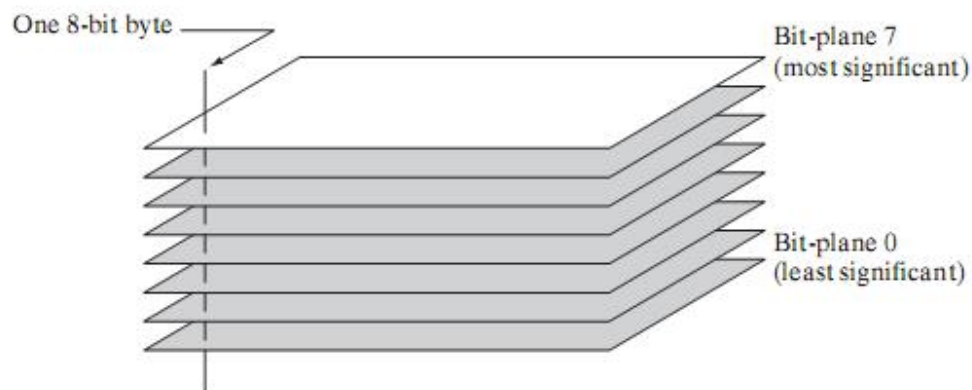


Fig.1.5 Bit-plane representation of an 8-bit image.

In terms of bit-plane extraction for an 8-bit image, it is not difficult to show that the (binary) image for bit-plane 7 can be obtained by processing the input image with a thresholding gray-level transformation function that (1) maps all levels in the image between 0 and 127 to one level (for example, 0); and (2) maps all levels between 129 and 255 to another (for example, 255).

Digital Image Processing

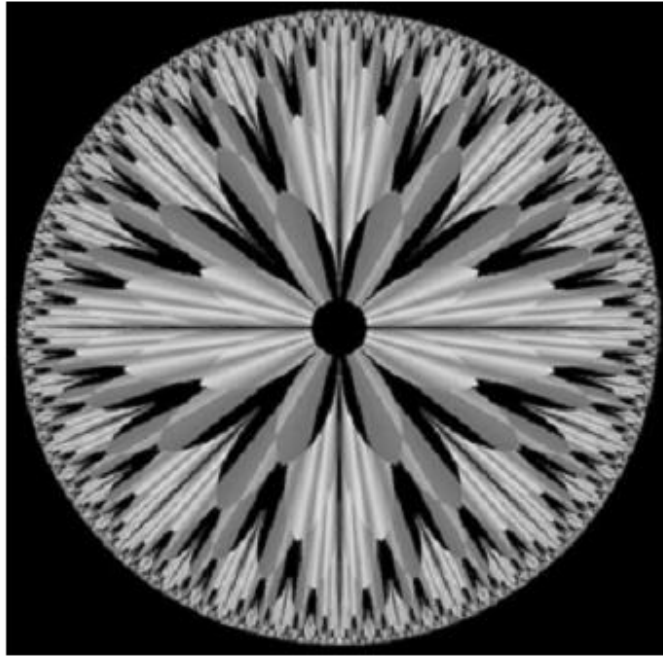


Fig.1.6 An 8-bit fractal image

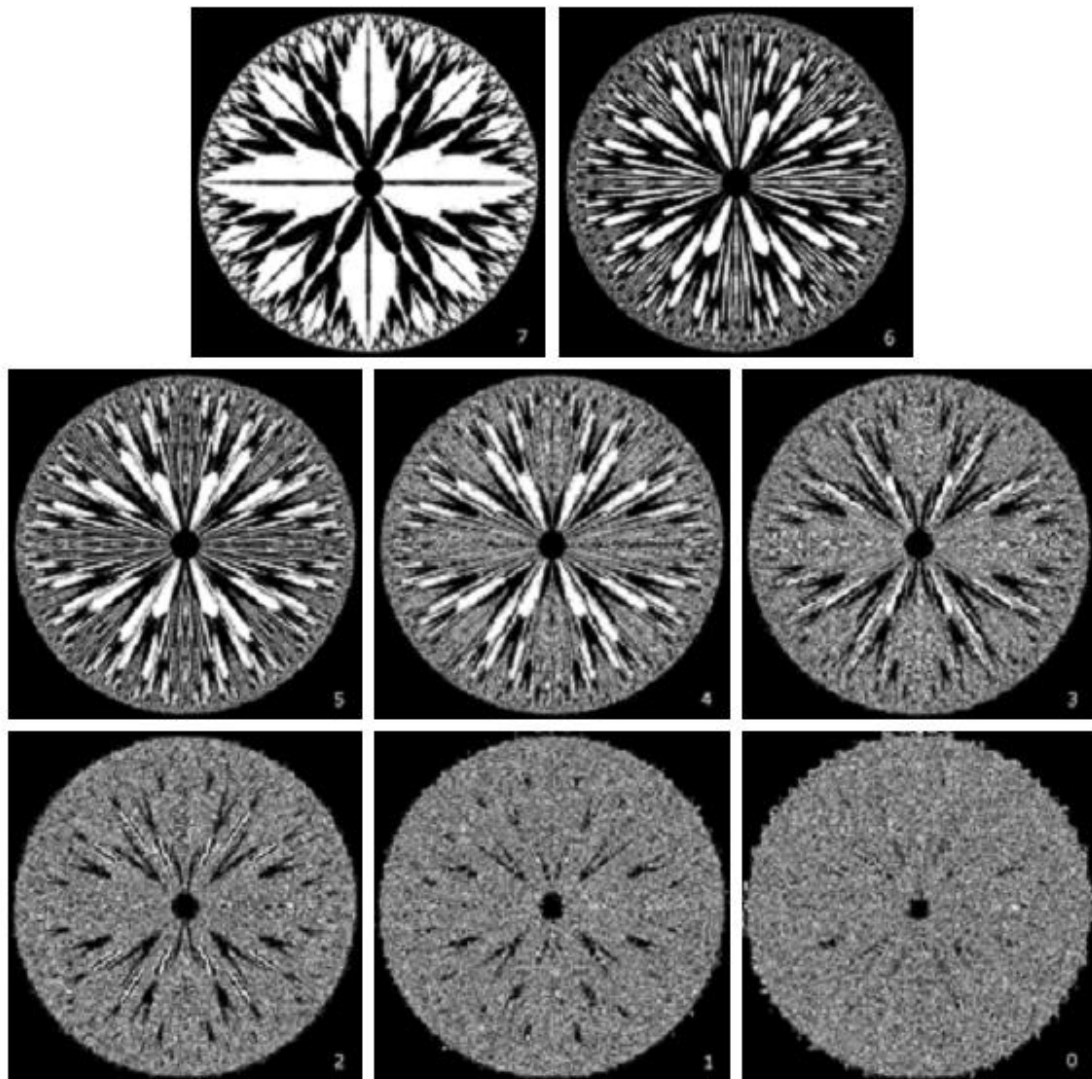


Fig.1.7 The eight bit planes of the image in Fig.1.6. The number at the bottom, right of each image identifies the bit plane.

Digital Image Processing

The term spatial domain refers to the aggregate of pixels composing an image. Spatial domain methods are procedures that operate directly on these pixels. Spatial domain processes will be denoted by the expression

$$g(x, y) = T[f(x, y)]$$

where $f(x, y)$ is the input image, $g(x, y)$ is the processed image, and T is an operator on f , defined over some neighborhood of (x, y) . In addition, T can operate on a set of input images, such as performing the pixel-by-pixel sum of K images for noise reduction.

The principal approach in defining a neighborhood about a point (x, y) is to use a square or rectangular subimage area centered at (x, y) , as Fig.2.1 shows. The center of the subimage is moved from pixel to pixel starting, say, at the top left corner. The operator T is applied at each location (x, y) to yield the output, g , at that location. The process utilizes only the pixels in the area of the image spanned by the neighborhood.

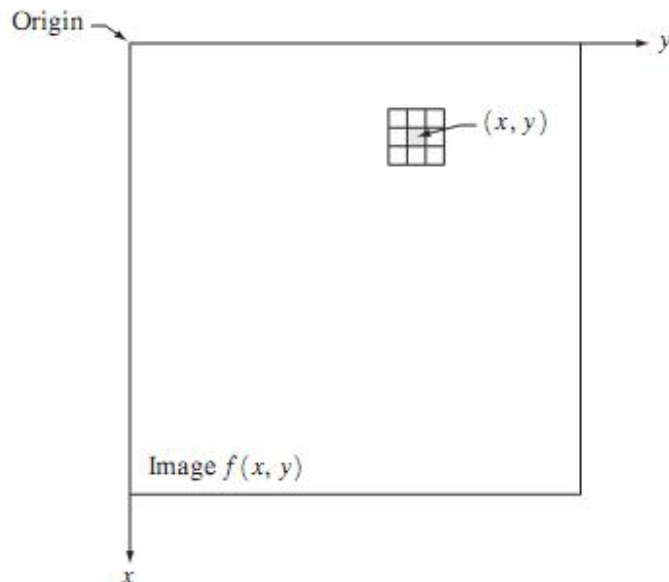


Fig.2.1 A 3*3 neighborhood about a point (x, y) in an image.

Although other neighborhood shapes, such as approximations to a circle, sometimes are used, square and rectangular arrays are by far the most predominant because of their ease of implementation. The simplest form of T is when the neighborhood is of size 1×1 (that is, a single pixel). In this case, g depends only on the value of f at (x, y) , and T becomes a gray-level (also called an intensity or mapping) transformation function of the form

$$s = T(r)$$

where, for simplicity in notation, r and s are variables denoting, respectively, the gray level of $f(x, y)$ and $g(x, y)$ at any point (x, y) . For example, if $T(r)$ has the form shown in Fig. 2.2(a), the effect of this transformation would be to produce an image of higher contrast than the original by darkening the levels below m and brightening the levels above m in the original image. In this technique, known as contrast stretching, the values of r below m are compressed by the transformation function into a narrow range of s , toward black. The opposite effect takes place for values of r above m . In the limiting case shown in Fig. 2.2(b), $T(r)$ produces a two-level (binary) image. A mapping of this form is called a thresholding function. Some fairly simple, yet powerful, processing approaches can be formulated with gray-level transformations. Because enhancement at any point in an image depends only on the gray level at that point, techniques in this category often are referred to as point processing.

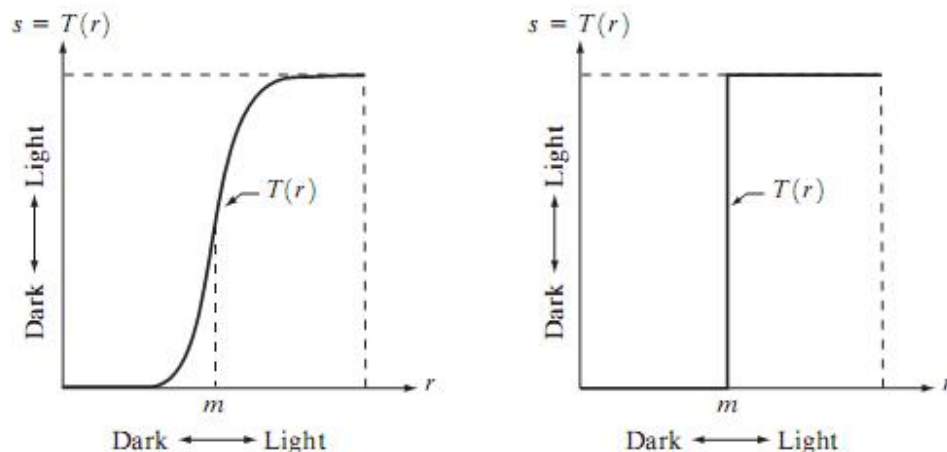


Fig.2.2 Graylevel transformation functions for contrast enhancement.

Larger neighborhoods allow considerably more flexibility. The general approach is to use a function of the values of f in a predefined neighborhood of (x, y) to determine the value of g at (x, y) . One of the principal approaches in this formulation is based on the use of so-called masks

(also referred to as filters, kernels, templates, or windows). Basically, a mask is a small (say, 3*3) 2-D array, such as the one shown in Fig. 2.1, in which the values of the mask coefficients determine the nature of the process, such as image sharpening.

Histogram Processing:

The histogram of a digital image with gray levels in the range $[0, L-1]$ is a discrete function $h(r_k) = (n_k)$, where r_k is the k th gray level and n_k is the number of pixels in the image having gray level r_k . It is common practice to normalize a histogram by dividing each of its values by the total number of pixels in the image, denoted by n . Thus, a normalized histogram is given by

$$p(r_k) = n_k/n$$

for $k=0,1,\dots,L-1$. Loosely speaking, $p(r_k)$ gives an estimate of the probability of occurrence of gray level r_k . Note that the sum of all components of a normalized histogram is equal to 1.

Histograms are the basis for numerous spatial domain processing techniques. Histogram manipulation can be used effectively for image enhancement. Histograms are simple to calculate in software and also lend themselves to economic hardware implementations, thus making them a popular tool for real-time image processing.

As an introduction to the role of histogram processing in image enhancement, consider Fig. 3, which is the pollen image shown in four basic gray-level characteristics: dark, light, low contrast, and high contrast. The right side of the figure shows the histograms corresponding to these images. The horizontal axis of each histogram plot corresponds to gray level values, r_k .

The vertical axis corresponds to values of $h(r_k) = n_k$ or $p(r_k) = n_k/n$ if the values are normalized. Thus, as indicated previously, these histogram plots are simply plots of $h(r_k) = n_k$ versus r_k or $p(r_k) = n_k/n$ versus r_k .

Digital Image Processing

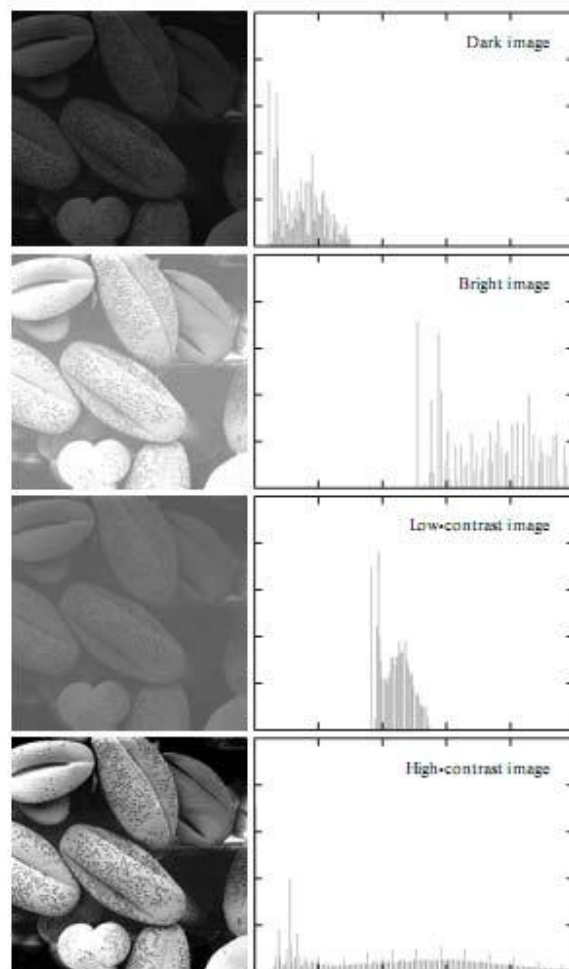


Fig.3 Four basic image types: dark, light, low contrast, high contrast, and their corresponding histograms.

We note in the dark image that the components of the histogram are concentrated on the low (dark) side of the gray scale. Similarly, the components of the histogram of the bright image are biased toward the high side of the gray scale. An image with low contrast has a histogram that will be narrow and will be centered toward the middle of the gray scale. For a monochrome image this implies a dull, washed-out gray look. Finally, we see that the components of the histogram in the high-contrast image cover a broad range of the gray scale and, further, that the distribution of pixels is not too far from uniform, with very few vertical lines being much higher than the others. Intuitively, it is reasonable to conclude that an image whose pixels tend to occupy the entire range of possible gray levels and, in addition, tend to be distributed uniformly, will have an appearance of high contrast and will exhibit a large variety of gray tones. The net effect will be an image that shows a great deal of gray-level detail and has high dynamic

Digital Image Processing

range. It will be shown shortly that it is possible to develop a transformation function that can automatically achieve this effect, based only on information available in the histogram of the input image.

Histogram Equalization:

Consider for a moment continuous functions, and let the variable r represent the gray levels of the image to be enhanced. We assume that r has been normalized to the interval $[0, 1]$, with $r=0$ representing black and $r=1$ representing white. Later, we consider a discrete formulation and allow pixel values to be in the interval $[0, L-1]$. For any r satisfying the aforementioned conditions, we focus attention on transformations of the form

$$s = T(r) \quad 0 \leq r \leq 1$$

that produce a level s for every pixel value r in the original image. For reasons that will become obvious shortly, we assume that the transformation function $T(r)$ satisfies the following conditions:

- (a) $T(r)$ is single-valued and monotonically increasing in the interval $0 \leq r \leq 1$; and
- (b) $0 \leq T(r) \leq 1$ for $0 \leq r \leq 1$.

The requirement in (a) that $T(r)$ be single valued is needed to guarantee that the inverse transformation will exist, and the monotonicity condition preserves the increasing order from black to white in the output image. A transformation function that is not monotonically increasing could result in at least a section of the intensity range being inverted, thus producing some inverted gray levels in the output image. Finally, condition (b) guarantees that the output gray levels will be in the same range as the input levels. Figure 4.1 gives an example of a transformation function that satisfies these two conditions. The inverse transformation from s back to r is denoted

$$r = T^{-1}(s) \quad 0 \leq s \leq 1.$$

It can be shown by example that even if $T(r)$ satisfies conditions (a) and (b), it is possible that the corresponding inverse $T^{-1}(s)$ may fail to be single valued.

Digital Image Processing

Fig.4.1 A gray-level transformation function that is both single valued and monotonically increasing.

The gray levels in an image may be viewed as random variables in the interval $[0, 1]$. One of the most fundamental descriptors of a random variable is its probability density function (PDF). Let $p_r(r)$ and $p_s(s)$ denote the probability density functions of random variables r and s , respectively, where the subscripts on p are used to denote that p_r and p_s are different functions. A basic result from an elementary probability theory is that, if $p_r(r)$ and $T(r)$ are known and $T^{-1}(s)$ satisfies condition (a), then the probability density function $p_s(s)$ of the transformed variable s can be obtained using a rather simple formula:

$$p_s(s) = p_r(r) \left| \frac{dr}{ds} \right|.$$

Thus, the probability density function of the transformed variable, s , is determined by the gray-level PDF of the input image and by the chosen transformation function. A transformation function of particular importance in image processing has the form

$$s = T(r) = \int_0^r p_r(w) dw$$

where w is a dummy variable of integration. The right side of Eq. above is recognized as the cumulative distribution function (CDF) of random variable r . Since probability density functions are always positive, and recalling that the integral of a function is the area under the function, it follows that this transformation function is single valued and monotonically increasing, and, therefore, satisfies condition (a). Similarly, the integral of a probability density function for variables in the range $[0, 1]$ also is in the range $[0, 1]$, so condition (b) is satisfied as well.

Digital Image Processing

Given transformation function $T(r)$, we find $p_s(s)$ by applying Eq. We know from basic calculus (Leibniz's rule) that the derivative of a definite integral with respect to its upper limit is simply the integrand evaluated at that limit. In other words,

$$\begin{aligned}\frac{ds}{dr} &= \frac{dT(r)}{dr} \\ &= \frac{d}{dr} \left[\int_0^r p_r(w) dw \right] \\ &= p_r(r).\end{aligned}$$

Substituting this result for dr/ds , and keeping in mind that all probability values are positive, yields

$$\begin{aligned}p_s(s) &= p_r(r) \left| \frac{dr}{ds} \right| \\ &= p_r(r) \left| \frac{1}{p_r(r)} \right| \\ &= 1 \quad 0 \leq s \leq 1.\end{aligned}$$

Because $p_s(s)$ is a probability density function, it follows that it must be zero outside the interval $[0, 1]$ in this case because its integral over all values of s must equal 1. We recognize the form of $p_s(s)$ as a uniform probability density function. Simply stated, we have demonstrated that performing the transformation function yields a random variable s characterized by a uniform probability density function. It is important to note from Eq. discussed above that $T(r)$ depends on $p_r(r)$, but, as indicated by Eq. after it, the resulting $p_s(s)$ always is uniform, independent of the form of $p_r(r)$. For discrete values we deal with probabilities and summations instead of probability density functions and integrals. The probability of occurrence of gray level r in an image is approximated by

$$p_r(r_k) = \frac{n_k}{n} \quad k = 0, 1, 2, \dots, L - 1$$

where, as noted at the beginning of this section, n is the total number of pixels in the image, n_k is the number of pixels that have gray level r_k , and L is the total number of possible gray levels in the image. The discrete version of the transformation function given in Eq. is

$$\begin{aligned}s_k &= T(r_k) = \sum_{j=0}^k p_r(r_j) \\ &= \sum_{j=0}^k \frac{n_j}{n} \quad k = 0, 1, 2, \dots, L - 1.\end{aligned}$$

Digital Image Processing

Thus, a processed (output) image is obtained by mapping each pixel with level r_k in the input image into a corresponding pixel with level s_k in the output image. As indicated earlier, a plot of $p_r(r_k)$ versus r_k is called a histogram. The transformation (mapping) is called histogram equalization or histogram linearization. It is not difficult to show that the transformation in Eq. satisfies conditions (a) and (b) stated previously. Unlike its continuous counterpart, it cannot be proved in general that this discrete transformation will produce the discrete equivalent of a uniform probability density function, which would be a uniform histogram.

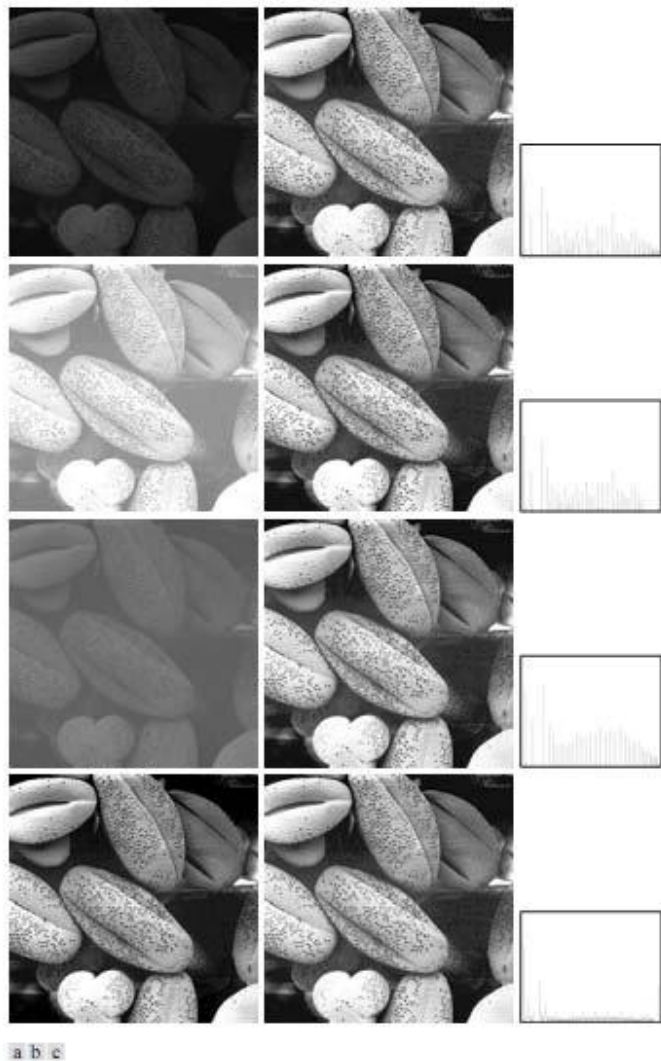


Fig.4.2 (a) Images from Fig.3 (b) Results of histogram equalization. (c) Corresponding histograms.

Digital Image Processing

The inverse transformation from s back to r is denoted by

$$r_k = T^{-1}(s_k) \quad k = 0, 1, 2, \dots, L - 1$$

Histogram Matching (Specification):

Histogram equalization automatically determines a transformation function that seeks to produce an output image that has a uniform histogram. When automatic enhancement is desired, this is a good approach because the results from this technique are predictable and the method is simple to implement. In particular, it is useful sometimes to be able to specify the shape of the histogram that we wish the processed image to have. The method used to generate a processed image that has a specified histogram is called histogram matching or histogram specification.

Development of the method:

Let us return for a moment to continuous gray levels r and z (considered continuous random variables), and let $p_r(r)$ and $p_z(z)$ denote their corresponding continuous probability density functions. In this notation, r and z denote the gray levels of the input and output (processed) images, respectively. We can estimate $p_r(r)$ from the given input image, while $p_z(z)$ is the specified probability density function that we wish the output image to have.

Let s be a random variable with the property

$$s = T(r) = \int_0^r p_r(w) dw$$

where w is a dummy variable of integration. We recognize this expression as the continuous version of histogram equalization. Suppose next that we define a random variable z with the property

$$G(z) = \int_0^z p_z(t) dt = s$$

Digital Image Processing

where t is a dummy variable of integration. It then follows from these two equations that $G(z)=T(r)$ and, therefore, that z must satisfy the condition

$$z = G^{-1}(s) = G^{-1}[T(r)].$$

The transformation $T(r)$ can be obtained once $p_r(r)$ has been estimated from the input image. Similarly, the transformation function $G(z)$ can be obtained because $p_z(z)$ is given. Assuming that G^{-1} exists and that it satisfies conditions (a) and (b) in the histogram equalization process, the above three equations show that an image with a specified probability density function can be obtained from an input image by using the following procedure:

- (1) Obtain the transformation function $T(r)$.
- (2) To obtain the transformation function $G(z)$.
- (3) Obtain the inverse transformation function G^{-1} .
- (4) Obtain the output image by applying above Eq. to all the pixels in the input image.

The result of this procedure will be an image whose gray levels, z , have the specified probability density function $p_z(z)$. Although the procedure just described is straightforward in principle, it is seldom possible in practice to obtain analytical expressions for $T(r)$ and for G^{-1} . Fortunately, this problem is simplified considerably in the case of discrete values. The price we pay is the same as in histogram equalization, where only an approximation to the desired histogram is achievable. In spite of this, however, some very useful results can be obtained even with crude approximations.

$$\begin{aligned} s_k &= T(r_k) = \sum_{j=0}^k p_r(r_j) \\ &= \sum_{j=0}^k \frac{n_j}{n} \quad k = 0, 1, 2, \dots, L - 1 \end{aligned}$$

where n is the total number of pixels in the image, n_j is the number of pixels with gray level r_j , and L is the number of discrete gray levels. Similarly, the discrete formulation is obtained from the given histogram $p_z(z_i)$, $i=0, 1, 2, \dots, L-1$, and has the form

$$v_k = G(z_k) = \sum_{i=0}^k p_z(z_i) = s_k \quad k = 0, 1, 2, \dots, L - 1.$$

Digital Image Processing

As in the continuous case, we are seeking values of z that satisfy this equation. The variable v_k was added here for clarity in the discussion that follows. Finally, the discrete version of the above Eqn. is given by

$$z_k = G^{-1}[T(r_k)] \quad k = 0, 1, 2, \dots, L - 1$$

Or

$$z_k = G^{-1}(s_k) \quad k = 0, 1, 2, \dots, L - 1.$$

Implementation:

We start by noting the following: (1) Each set of gray levels $\{r_j\}$, $\{s_j\}$, and $\{z_j\}$, $j=0, 1, 2, \dots, L-1$, is a one-dimensional array of dimension $L \times 1$. (2) All mappings from r to s and from s to z are simple table lookups between a given pixel value and these arrays. (3) Each of the elements of these arrays, for example, s_k , contains two important pieces of information: The subscript k denotes the location of the element in the array, and s denotes the value at that location. (4) We need to be concerned only with integer pixel values. For example, in the case of an 8-bit image, $L=256$ and the elements of each of the arrays just mentioned are integers between 0 and 255. This implies that we now work with gray level values in the interval $[0, L-1]$ instead of the normalized interval $[0, 1]$ that we used before to simplify the development of histogram processing techniques.

In order to see how histogram matching actually can be implemented, consider Fig. 5(a), ignoring for a moment the connection shown between this figure and Fig. 5(c). Figure 5(a) shows a hypothetical discrete transformation function $s=T(r)$ obtained from a given image. The first gray level in the image, r_1 , maps to s_1 ; the second gray level, r_2 , maps to s_2 ; the k th level r_k maps to s_k ; and so on (the important point here is the ordered correspondence between these values). Each value s_j in the array is precomputed, so the process of mapping simply uses the actual value of a pixel as an index in an array to determine the corresponding value of s . This process is particularly easy because we are dealing with integers. For example, the s mapping for an 8-bit pixel with value 127 would be found in the 128th position in array $\{s_j\}$ (recall that we start at 0) out of the possible 256 positions. If we stopped here and mapped the value of each pixel of an input image by the

Digital Image Processing

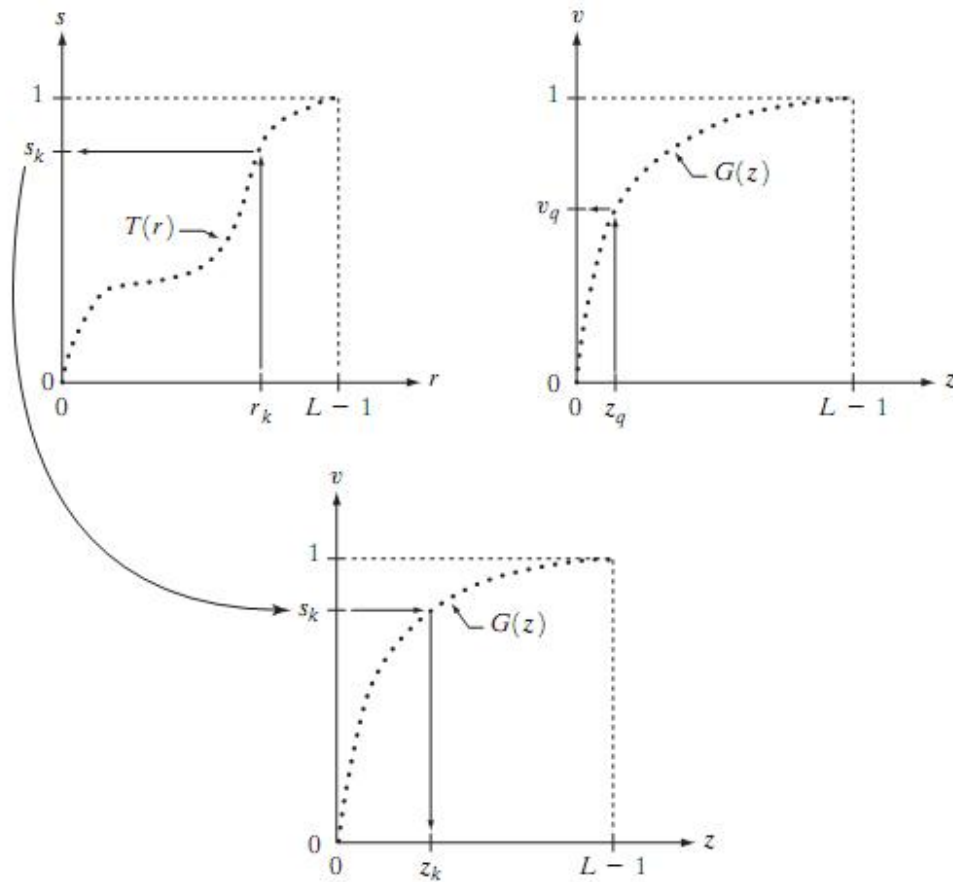


Fig.5. (a) Graphical interpretation of mapping from r_k to s_k via $T(r)$. (b) Mapping of z_q to its corresponding value v_q via $G(z)$ (c) Inverse mapping from s_k to its corresponding value of z_k .

method just described, the output would be a histogram-equalized image. In order to implement histogram matching we have to go one step further. Figure 5(b) is a hypothetical transformation function G obtained from a given histogram $p_z(z)$. For any z_q , this transformation function yields a corresponding value v_q . This mapping is shown by the arrows in Fig. 5(b). Conversely, given any value v_q , we would find the corresponding value z_q from G^{-1} . In terms of the figure, all this means graphically is that we would reverse the direction of the arrows to map v_q into its corresponding z_q . However, we know from the definition that $v=s$ for corresponding subscripts, so we can use exactly this process to find the z_k corresponding to any value s_k that we computed previously from the equation $s_k = T(r_k)$. This idea is shown in Fig.5(c).

Digital Image Processing

Since we really do not have the z 's (recall that finding these values is precisely the objective of histogram matching), we must resort to some sort of iterative scheme to find z from s . The fact that we are dealing with integers makes this a particularly simple process. Basically, because $v_k = s_k$, we have that the z 's for which we are looking must satisfy the equation $G(z_k) = s_k$, or $(G(z_k) - s_k) = 0$. Thus, all we have to do to find the value of z_k corresponding to s_k is to iterate on values of z such that this equation is satisfied for $k=0, 1, 2, \dots, L-1$. We do not have to find the inverse of G because we are going to iterate on z . Since we are dealing with integers, the closest we can get to satisfying the equation $(G(z_k) - s_k) = 0$ is to let $z_k = \hat{z}$ for each value of k , where $\hat{z}$ is the smallest integer in the interval $[0, L-1]$ such that

$$(G(\hat{z}) - s_k) \geq 0 \quad k = 0, 1, 2, \dots, L-1.$$

Given a value s_k , all this means conceptually in terms of Fig. 5(c) is that we would start with and increase it in integer steps until Eq is satisfied, at which point we let repeating this process for all values of k would yield all the required mappings from s to z , which constitutes the implementation of Eq. In practice, we would not have to start with each time because the values of s_k are known to increase monotonically. Thus, for $k=k+1$, we would start with $\hat{z} = z_k$ and increment in integer values from there.

Local Enhancement:

The histogram processing methods discussed in the previous two sections are global, in the sense that pixels are modified by a transformation function based on the gray-level content of an entire image. Although this global approach is suitable for overall enhancement, there are cases in which it is necessary to enhance details over small areas in an image. The number of pixels in these areas may have negligible influence on the computation of a global transformation whose shape does not necessarily guarantee the desired local enhancement. The solution is to devise transformation functions based on the gray-level distribution—or other properties—in the neighborhood of every pixel in the image.

The histogram processing techniques are easily adaptable to local enhancement. The procedure is to define a square or rectangular neighborhood and move the center of this area from pixel to pixel. At each location, the histogram of the points in the neighborhood is computed and either a histogram equalization or histogram specification transformation function is obtained. This function is finally used to map the gray level of the

Digital Image Processing

pixel centered in the neighborhood. The center of the neighborhood region is then moved to an adjacent pixel location and the procedure is repeated. Since only one new row or column of the neighborhood changes during a pixel-to-pixel translation of the region, updating the histogram obtained in the previous location with the new data introduced at each motion step is possible. This approach has obvious advantages over repeatedly computing the histogram over all pixels in the neighborhood region each time the region is moved one pixel location. Another approach used some times to reduce computation is to utilize nonoverlapping regions, but this method usually produces an undesirable checkerboard effect.

Image Subtraction:

The difference between two images $f(x, y)$ and $h(x, y)$, expressed as

$$g(x, y) = f(x, y) - h(x, y),$$

is obtained by computing the difference between all pairs of corresponding pixels from f and h . The key usefulness of subtraction is the enhancement of differences between images. The higher-order bit planes of an image carry a significant amount of visually relevant detail, while the lower planes contribute more to fine (often imperceptible) detail. Figure 7(a) shows the fractal image used earlier to illustrate the concept of bit planes. Figure 7(b) shows the result of discarding (setting to zero) the four least significant bit planes of the original image. The images are nearly identical visually, with the exception of a very slight drop in overall contrast due to less variability of the graylevel values in the image of Fig. 7(b). The pixel-by-pixel difference between these two images is shown in Fig. 7(c). The differences in pixel values are so small that the difference image appears nearly black when displayed on an 8-bit display. In order to bring out more detail, we can perform a contrast stretching transformation. We chose histogram equalization, but an appropriate power-law transformation would have done the job also. The result is shown in Fig. 7(d). This is a very useful image for evaluating the effect of setting to zero the lower-order planes.

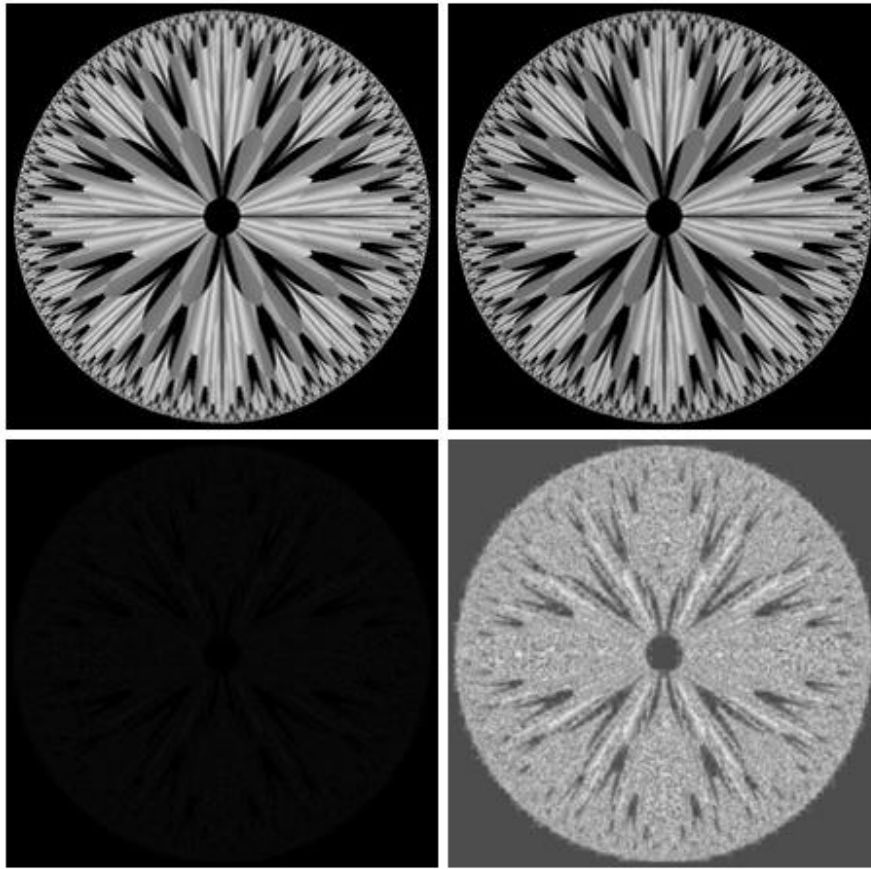


Fig.7 (a) Original fractal image (b) Result of setting the four lower-order bit planes to zero (c) Difference between (a) and(b) (d) Histogram equalized difference image.

One of the most commercially successful and beneficial uses of image subtraction is in the area of medical imaging called mask mode radiography. In this case $h(x, y)$, the mask, is an X-ray image of a region of a patient's body captured by an intensified TV camera (instead of traditional X-ray film) located opposite an X-ray source. The procedure consists of injecting a contrast medium into the patient's bloodstream, taking a series of images of the same anatomical region as $h(x, y)$, and subtracting this mask from the series of incoming images after injection of the contrast medium. The net effect of subtracting the mask from each sample in the incoming stream of TV images is that the areas that are different between $f(x, y)$ and $h(x, y)$ appear in the output image as enhanced detail. Because images can be captured at TV rates, this procedure in essence gives a movie showing how the contrast medium propagates through the various arteries in the area being observed.

Digital Image Processing

Image Averaging:

Consider a noisy image $g(x, y)$ formed by the addition of noise $h(x, y)$ to an original image $f(x, y)$; that is,

$$g(x, y) = f(x, y) + \eta(x, y)$$

where the assumption is that at every pair of coordinates (x, y) the noise is uncorrelated and has zero average value. The objective of the following procedure is to reduce the noise content by adding a set of noisy images, $\{g_i(x, y)\}$. If the noise satisfies the constraints just stated, it can be shown that if an image $\bar{g}(x, y)$ is formed by averaging K different noisy images,

$$\bar{g}(x, y) = \frac{1}{K} \sum_{i=1}^K g_i(x, y)$$

Then it follows that

$$E\{\bar{g}(x, y)\} = f(x, y)$$

and

$$\sigma_{\bar{g}(x, y)}^2 = \frac{1}{K} \sigma_{\eta(x, y)}^2$$

Where $E\{\bar{g}(x, y)\}$ is the expected value of $\bar{g}$ and $\sigma_{\bar{g}(x, y)}^2$ and $\sigma_{\eta(x, y)}^2$ are the variances of $\bar{g}$ and η , all at coordinates (x, y) . The standard deviation at any point in the average image is

$$\sigma_{\bar{g}(x, y)} = \frac{1}{\sqrt{K}} \sigma_{\eta(x, y)}$$

Digital Image Processing

As K increases, the above equations indicate that the variability (noise) of the pixel values at each location (x, y) decreases. Because $E\{\bar{g}(x, y)\} = f(x, y)$, this means that $\bar{g}(x, y)$ approaches $f(x, y)$ as the number of noisy images used in the averaging process increases. In practice, the images $g_i(x, y)$ must be registered (aligned) in order to avoid the introduction of blurring and other artifacts in the output image

Basics of Spatial Filtering:

Some neighborhood operations work with the values of the image pixels in the neighborhood and the corresponding values of a subimage that has the same dimensions as the neighborhood. The subimage is called a filter, mask, kernel, template, or window, with the first three terms being the most prevalent terminology. The values in a filter subimage are referred to as coefficients, rather than pixels. The concept of filtering has its roots in the use of the Fourier transform for signal processing in the so-called frequency domain. We use the term spatial filtering to differentiate this type of process from the more traditional frequency domain filtering.

The mechanics of spatial filtering are illustrated in Fig. 9.1. The process consists simply of moving the filter mask from point to point in an image. At each point (x, y) , the response of the filter at that point is calculated using a predefined relationship. The response is given by a sum of products of the filter coefficients and the corresponding image pixels in the area spanned by the filter mask. For the 3×3 mask shown in Fig. 9.1, the result (or response), R , of linear filtering with the filter mask at a point (x, y) in the image is

$$R = w(-1, -1)f(x - 1, y - 1) + w(-1, 0)f(x - 1, y) + \cdots \\ + w(0, 0)f(x, y) + \cdots + w(1, 0)f(x + 1, y) + w(1, 1)f(x + 1, y + 1),$$

which we see is the sum of products of the mask coefficients with the corresponding pixels directly under the mask. Note in particular that the coefficient $w(0, 0)$ coincides with image

Digital Image Processing

value $f(x, y)$, indicating that the mask is centered at (x, y) when the computation of the sum of products takes place. For a mask of size $m \times n$, we assume that $m=2a+1$ and $n=2b+1$, where a and b are nonnegative integers.

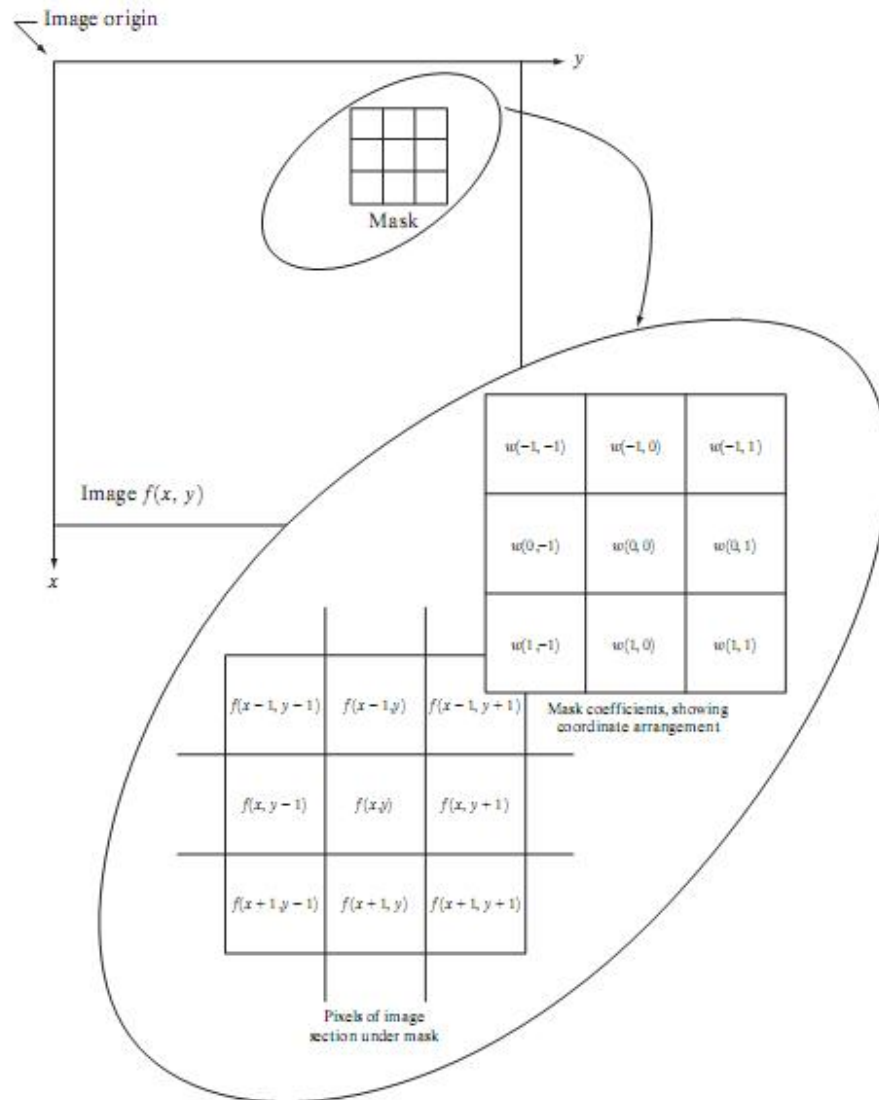


Fig.9.1 The mechanics of spatial filtering. The magnified drawing shows a 3X3 mask and the image section directly under it; the image section is shown displaced out from under the mask for ease of readability.

In general, linear filtering of an image f of size $M \times N$ with a filter mask of size $m \times n$ is given by the expression:

Digital Image Processing

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

where, from the previous paragraph, $a=(m-1)/2$ and $b=(n-1)/2$. To generate a complete filtered image this equation must be applied for $x=0,1,2,\dots, M-1$ and $y=0,1,2,\dots, N-1$. In this way, we are assured that the mask processes all pixels in the image. It is easily verified when $m=n=3$ that this expression reduces to the example given in the previous paragraph.

The process of linear filtering is similar to a frequency domain concept called convolution. For this reason, linear spatial filtering often is referred to as “convolving a mask with an image.” Similarly, filter masks are sometimes called convolution masks. The term convolution kernel also is in common use. When interest lies on the response, R , of an $m \times n$ mask at any point (x,y) , and not on the mechanics of implementing mask convolution, it is common practice to simplify the notation by using the following expression:

$$\begin{aligned} R &= w_1 z_1 + w_2 z_2 + \dots + w_{mn} z_{mn} \\ &= \sum_{i=1}^{mn} w_i z_i \end{aligned}$$

where the w 's are mask coefficients, the z 's are the values of the image graylevels corresponding to those coefficients, and mn is the total number of coefficients in the mask. For the 3×3 general mask shown in Fig.9.2 the response at any point (x, y) in the image is given by

$$\begin{aligned} R &= w_1 z_1 + w_2 z_2 + \dots w_9 z_9 \\ &= \sum_{i=1}^9 w_i z_i. \end{aligned}$$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

Fig.9.2 Another representation of a general 3 x 3 spatial filter mask.

An important consideration in implementing neighborhood operations for spatial filtering is the issue of what happens when the center of the filter approaches the border of the image. Consider for simplicity a square mask of size $n \times n$. At least one edge of such a mask will coincide with the border of the image when the center of the mask is at a distance of $(n-1)/2$ pixels away from the border of the image. If the center of the mask moves any closer to the border, one or more rows or columns of the mask will be located outside the image plane. There are several ways to handle this situation. The simplest is to limit the excursions of the center of the mask to be at a distance no less than $(n-1)/2$ pixels from the border. The resulting filtered image will be smaller than the original, but all the pixels in the filtered image will have been processed with the full mask. If the result is required to be the same size as the original, then the approach typically employed is to filter all pixels only with the section of the mask that is fully contained in the image. With this approach, there will be bands of pixels near the border that will have been processed with a partial filter mask. Other approaches include “padding” the image by adding rows and columns of 0’s (or other constant gray level), or padding by replicating rows or columns. The padding is then stripped off at the end of the process.

This keeps the size of the filtered image the same as the original, but the values of the padding will have an effect near the edges that becomes more prevalent as the size of the mask increases. The only way to obtain a perfectly filtered result is to accept a somewhat smaller filtered image by limiting the excursions of the center of the filter mask to a distance no less than $(n-1)/2$ pixels from the border of the original image.

Image smoothing

Smoothing Spatial Filters:

Smoothing filters are used for blurring and for noise reduction. Blurring is used in preprocessing steps, such as removal of small details from an image prior to (large) object extraction, and bridging of small gaps in lines or curves. Noise reduction can be accomplished by blurring with a linear filter and also by non-linear filtering.

(1) Smoothing Linear Filters:

The output (response) of a smoothing, linear spatial filter is simply the average of the pixels contained in the neighborhood of the filter mask. These filters sometimes are called averaging filters. The idea behind smoothing filters is straightforward. By replacing the value of every pixel

Digital Image Processing

in an image by the average of the gray levels in the neighborhood defined by the filter mask, this process results in an image with reduced “sharp” transitions in gray levels. Because random noise typically consists of sharp transitions in gray levels, the most obvious application of smoothing is noise reduction. However, edges (which almost always are desirable features of an image) also are characterized by sharp transitions in gray levels, so averaging filters have the undesirable side effect that they blur edges. Another application of this type of process includes the smoothing of false contours that result from using an insufficient number of gray levels.

$$\frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \frac{1}{16} \times \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Fig.10.1 Two 3 x 3 smoothing (averaging) filter masks.The constant multiplier in front of each mask is equal to the sum of the values of its coefficients, as is required to compute an average.

A major use of averaging filters is in the reduction of “irrelevant” detail in an image. By “irrelevant” we mean pixel regions that are small with respect to the size of the filter mask.

Figure 10.1 shows two 3 x 3 smoothing filters. Use of the first filter yields the standard average of the pixels under the mask. This can best be seen by substituting the coefficients of the mask in

$$R = \frac{1}{9} \sum_{i=1}^9 z_i,$$

which is the average of the gray levels of the pixels in the 3 x 3 neighborhood defined by the mask. Note that, instead of being 1/9, the coefficients of the filter are all 1's. The idea here is that it is computationally more efficient to have coefficients valued 1. At the end of the filtering process the entire image is divided by 9. An m x n mask would have a normalizing constant equal to 1/mn.

A spatial averaging filter in which all coefficients are equal is sometimes called a box filter.

The second mask shown in Fig.10.1 is a little more interesting. This mask yields a so-called weighted average, terminology used to indicate that pixels are multiplied by different coefficients, thus giving more importance (weight) to some pixels at the expense of others. In the mask shown in Fig. 10.1(b) the pixel at the center of the mask is multiplied by a higher value than any other, thus giving this pixel more importance in the calculation of the average. The other pixels are inversely weighted as a function of their distance from the center of the mask. The diagonal terms are further away from the center than the orthogonal neighbors (by a factor of $\sqrt{2}$) and, thus, are weighed less than these immediate neighbors of the center pixel. The basic strategy behind weighing the center point the highest and then reducing the value of the coefficients as a function of increasing distance from the origin is simply an attempt to reduce blurring in the smoothing process. We could have picked other weights to accomplish the same general objective. However, the sum of all the coefficients in the mask of Fig. 10.1(b) is equal to 16, an attractive feature for computer implementation because it has an integer power of 2. In practice, it is difficult in general to see differences between images smoothed by using either of the masks in Fig. 10.1, or similar arrangements, because the area these masks span at any one location in an image is so small.

The general implementation for filtering an $M \times N$ image with a weighted averaging filter of size $m \times n$ (m and n odd) is given by the expression

$$g(x, y) = \frac{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)}{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t)}$$

(2) Order-Statistics Filters:

Order-statistics filters are nonlinear spatial filters whose response is based on ordering (ranking) the pixels contained in the image area encompassed by the filter, and then replacing the value of the center pixel with the value determined by the ranking result. The best-known example in this category is the median filter, which, as its name implies, replaces the value of a pixel by the median of the gray levels in the neighborhood of that pixel (the original value of the pixel is included in the computation of the median). Median filters are quite popular because, for certain types of random noise, they provide excellent noise-reduction capabilities, with considerably less blurring than linear smoothing filters of similar size. Median filters are particularly effective in the presence of impulse noise, also called salt-and-pepper noise because of its appearance as white and black dots superimposed on an image.

The median, ϵ , of a set of values is such that half the values in the set are less than or equal to ϵ , and half are greater than or equal to ϵ . In order to perform median filtering at a point in an image, we first sort the values of the pixel in question and its neighbors, determine their median, and

Digital Image Processing

assign this value to that pixel. For example, in a 3 x 3 neighborhood the median is the 5th largest value, in a 5 x 5 neighborhood the 13th largest value, and so on. When several values in a neighborhood are the same, all equal values are grouped. For example, suppose that a 3 x 3 neighborhood has values (10, 20, 20, 20, 15, 20, 20, 25, 100). These values are sorted as (10, 15, 20, 20, 20, 20, 20, 25, 100), which results in a median of 20. Thus, the principal function of median filters is to force points with distinct gray levels to be more like their neighbors. In fact, isolated clusters of pixels that are light or dark with respect to their neighbors, and whose area is less than $n^2 / 2$ (one-half the filter area), are eliminated by an $n \times n$ median filter. In this case “eliminated” means forced to the median intensity of the neighbors. Larger clusters are affected considerably less

Use of Second Derivatives for Enhancement–The Laplacian:

The approach basically consists of defining a discrete formulation of the second-order derivative and then constructing a filter mask based on that formulation. We are interested in isotropic filters, whose response is independent of the direction of the discontinuities in the image to which the filter is applied. In other words, isotropic filters are rotation invariant, in the sense that rotating the image and then applying the filter gives the same result as applying the filter to the image first and then rotating the result.

Development of the method:

It can be shown (Rosenfeld and Kak [1982]) that the simplest isotropic derivative operator is the Laplacian, which, for a function (image) $f(x, y)$ of two variables, is defined as

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}.$$

Because derivatives of any order are linear operations, the Laplacian is a linear operator. In order to be useful for digital image processing, this equation needs to be expressed in discrete form. There are several ways to define a digital Laplacian using neighborhoods. digital second. Taking into account that we now have two variables, we use the following notation for the partial second-order derivative in the x-direction:

Digital Image Processing

$$\frac{\partial^2 f}{\partial^2 x^2} = f(x + 1, y) + f(x - 1, y) - 2f(x, y)$$

and, similarly in the y-direction, as

$$\frac{\partial^2 f}{\partial^2 y^2} = f(x, y + 1) + f(x, y - 1) - 2f(x, y)$$

The digital implementation of the two-dimensional Laplacian in Eq. is obtained by summing these two components

$$\nabla^2 f = [f(x + 1, y) + f(x - 1, y) + f(x, y + 1) + f(x, y - 1)] - 4f(x, y). \quad ($$

This equation can be implemented using the mask shown in Fig.11.1(a), which gives an isotropic result for rotations in increments of 90°.

The diagonal directions can be incorporated in the definition of the digital Laplacian by adding two more terms to Eq., one for each of the two diagonal directions. The form of each new term is the same as either Eq.

Digital Image Processing

0	1	0	1	1	1
1	-4	1	1	-8	1
0	1	0	1	1	1

0	-1	0	-1	-1	-1
-1	4	-1	-1	8	-1
0	-1	0	-1	-1	-1

a

b

c

d

Fig.11.1. (a) Filter mask used to implement the digital Laplacian (b) Mask used to implement an extension of this equation that includes the diagonal neighbors. (c) and (d) Two other implementations of the Laplacian.

but the coordinates are along the diagonals. Since each diagonal term also contains a $-2f(x, y)$ term, the total subtracted from the difference terms now would be $-8f(x, y)$. The mask used to implement this new definition is shown in Fig.11.1(b). This mask yields isotropic results for increments of 45° . The other two masks shown in Fig. 11 also are used frequently in practice.

They are based on a definition of the Laplacian that is the negative of the one we used here. As such, they yield equivalent results, but the difference in sign must be kept in mind when combining (by addition or subtraction) a Laplacian-filtered image with another image.

Because the Laplacian is a derivative operator, its use highlights gray-level discontinuities in an image and deemphasizes regions with slowly varying gray levels. This will tend to produce images that have grayish edge lines and other discontinuities, all superimposed on a dark, featureless background. Background features can be “recovered” while still preserving the sharpening effect of the Laplacian operation simply by adding the original and Laplacian images. As noted in the previous paragraph, it is important to keep in mind which definition of the Laplacian is used. If the definition used has a negative center coefficient, then we subtract, rather

than add, the Laplacian image to obtain a sharpened result. Thus, the basic way in which we use the Laplacian for image enhancement is as follows:

$$g(x, y) = \begin{cases} f(x, y) - \nabla^2 f(x, y) & \text{if the center coefficient of the} \\ & \text{Laplacian mask is negative} \\ f(x, y) + \nabla^2 f(x, y) & \text{if the center coefficient of the} \\ & \text{Laplacian mask is positive.} \end{cases}$$

Use of First Derivatives for Enhancement—The Gradient:

First derivatives in image processing are implemented using the magnitude of the gradient. For a function $f(x, y)$, the gradient of f at coordinates (x, y) is defined as the two-dimensional column vector

$$\nabla f = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}.$$

The magnitude of this vector is given by

$$\begin{aligned} \nabla f &= \text{mag}(\nabla f) \\ &= [G_x^2 + G_y^2]^{1/2} \\ &= \left[\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2 \right]^{1/2}. \end{aligned}$$

The components of the gradient vector itself are linear operators, but the magnitude of this vector obviously is not because of the squaring and square root operations. On the other hand, the partial derivatives are not rotation invariant (isotropic), but the magnitude of the gradient vector is. Although it is not strictly correct, the magnitude of the gradient vector often is referred to as the gradient.

Digital Image Processing

The computational burden of implementing over an entire image is not trivial, and it is common practice to approximate the magnitude of the gradient by using absolute values instead of squares and square roots:

$$\nabla f \approx |G_x| + |G_y|.$$

This equation is simpler to compute and it still preserves relative changes in gray levels, but the isotropic feature property is lost in general. However, as in the case of the Laplacian, the isotropic properties of the digital gradient defined in the following paragraph are preserved only for a limited number of rotational increments that depend on the masks used to approximate the derivatives. As it turns out, the most popular masks used to approximate the gradient give the same result only for vertical and horizontal edges and thus the isotropic properties of the gradient are preserved only for multiples of 90°.

As in the case of the Laplacian, we now define digital approximations to the preceding equations, and from there formulate the appropriate filter masks. In order to simplify the discussion that follows, we will use the notation in Fig. 11.2 (a) to denote image points in a 3 x 3 region. For example, the center point, z_5 , denotes $f(x, y)$, z_1 denotes $f(x-1, y-1)$, and so on. The simplest approximations to a first-order derivative that satisfy the conditions stated in that section are $G_x = (z_8 - z_5)$ and $G_y = (z_6 - z_5)$. Two other definitions proposed by Roberts [1965] in the early development of digital image processing use cross differences:

$$G_x = (z_9 - z_5) \quad \text{and} \quad G_y = (z_8 - z_6),$$

we compute the gradient as

$$\nabla f = [(z_9 - z_5)^2 + (z_8 - z_6)^2]^{1/2}$$

If we use absolute values, then substituting the quantities in the equations gives us the following approximation to the gradient:

$$\nabla f \approx |z_9 - z_5| + |z_8 - z_6|.$$

This equation can be implemented with the two masks shown in Figs. 11.2 (b) and(c). These masks are referred to as the Roberts cross-gradient operators. Masks of even size are awkward to implement. The smallest filter mask in which we are interested is of size 3 x 3. An approximation using absolute values, still at point z_5 , but using a 3*3 mask, is

$$\nabla f \approx |(z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)| \\ + |(z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)|.$$

Digital Image Processing

The difference between the third and first rows of the 3 x 3 image region approximates the derivative in the x-direction, and the difference between the third and first columns approximates the derivative in the y-direction. The masks shown in Figs. 11.2 (d) and (e), called the Sobel operators. The idea behind using a weight value of 2 is to achieve some smoothing by giving more importance to the center point. Note that the coefficients in all the masks shown in Fig. 11.2 sum to 0, indicating that they would give a response of 0 in an area of constant gray level, as expected of a derivative operator.

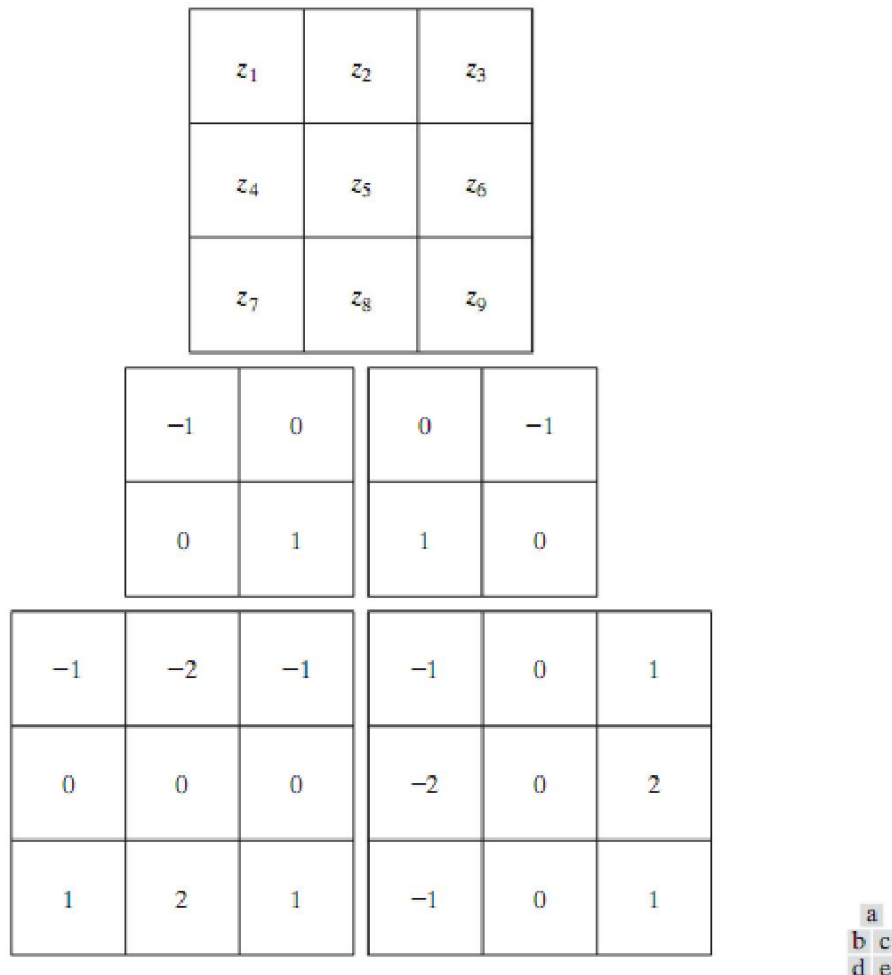


Fig.11.2 A 3 x 3 region of an image (the z 's are gray-level values) and masks used to compute the gradient at point labeled z_5 . All masks coefficients sum to zero, as expected of a derivative operator.

Enhancement In Frequency Domain:

The frequency domain methods of image enhancement are based on convolution theorem. This is represented as,

$$g(x, y) = h(x, y) * f(x, y)$$

Where.

$g(x, y)$ = Resultant image

$h(x, y)$ = Position invariant operator

$f(x, y)$ = Input image

The Fourier transform representation of equation above is,

$$G(u, v) = H(u, v) F(u, v)$$

The function $H(u, v)$ in equation is called transfer function. It is used to boost the edges of input image $f(x, y)$ to emphasize the high frequency components.

The different frequency domain methods for image enhancement are as follows.

- (c) Contrast stretching.
- (d) Clipping and thresholding.
- (e) Digital negative.
- (f) Intensity level slicing and
- (g) Bit extraction.

1. Contrast Stretching:

Due to non-uniform lighting conditions, there may be poor contrast between the background and the feature of interest. Figure 1.1 (a) shows the contrast stretching transformations.

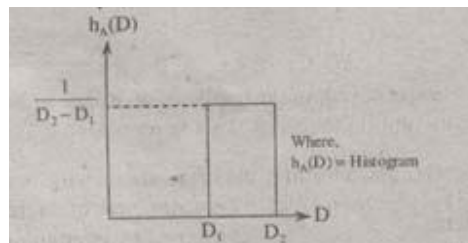


Fig.1.1 (a) Histogram of input image

Digital Image Processing

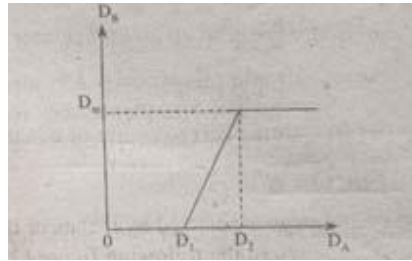


Fig.1.1 (b) Linear Law

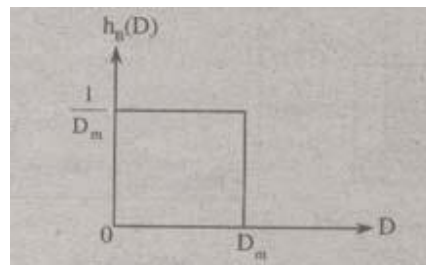


Fig.1.1 (c) Histogram of the transformed image

These stretching transformations are expressed as

In the area of stretching the slope of transformation is considered to be greater than unity. The parameters of stretching transformations i.e., a and b can be determined by examining the histogram of the image.

2. Clipping and Thresholding:

Clipping is considered as the special scenario of contrast stretching. It is the case in which the parameters are $\alpha = \gamma = 0$. Clipping is more advantageous for reduction of noise in input signals of range $[a, b]$.

Threshold of an image is selected by means of its histogram. Let us take the image shown in the following figure 1.2.

Digital Image Processing

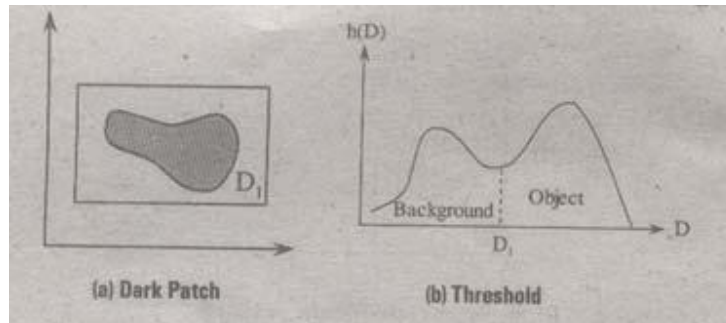


Fig. 1.2

The figure 1.2 (b) consists of two peaks i.e., background and object. At the abscissa of histogram minimum (D_1) the threshold is selected. This selected threshold (D_1) can separate background and object to convert the image into its respective binary form. The thresholding transformations are shown in figure 1.3.

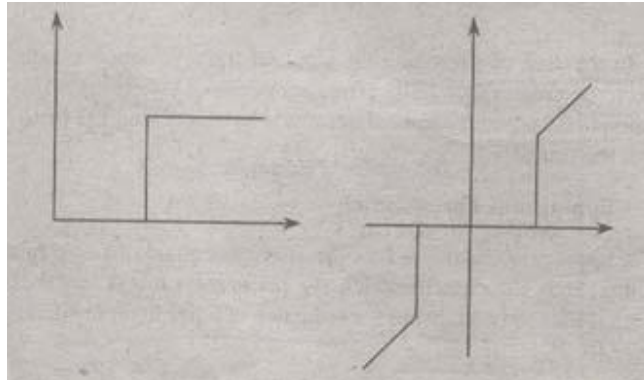


Fig.1.3

3. Digital Negative:

The digital negative of an image is achieved by reverse scaling of its grey levels to the transformation. They are much essential in displaying of medical images.

A digital negative transformation of an image is shown in figure 1.4.

Digital Image Processing

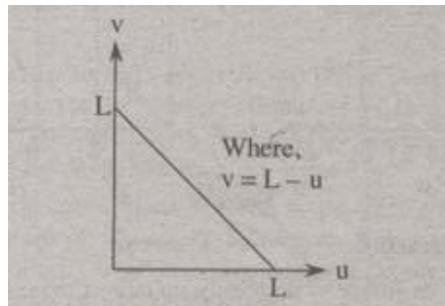


Fig.1.4

4. Intensity Level Slicing:

The images which consist of grey levels in between intensity at background and other objects require to reduce the intensity of the object. This process of changing intensity level is done with the help of intensity level slicing. They are expressed as

$$V = \begin{cases} L, & a \leq u \leq b \\ 0, & \text{elsewhere} \end{cases} \quad \text{without background}$$

And

$$V = \begin{cases} L, & a \leq u \leq b \\ u, & \text{elsewhere} \end{cases} \quad \text{with background}$$

The histogram of input image and its respective intensity level slicing is shown in the figure 1.5.

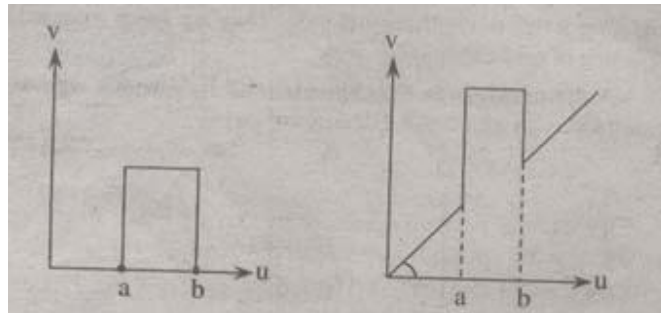


Fig.1.5

Digital Image Processing

When an image is uniformly quantized then, the n^{th} most significant bit can be extracted and displayed.

$$\text{Let, } u = k_1 2^{B-1} + k_2 2^{B-2} + \dots + k_{B-1} 2 + k_B$$

Then, the output is expressed as

$$V = \begin{cases} L, & \text{for } k_n = 1 \\ 0, & \text{elsewhere} \end{cases}$$

The spatial domain refers to the image plane itself, and approaches in this category are based on direct manipulation of pixels in an image. Frequency domain processing techniques are based on modifying the Fourier transform of an image.

The term spatial domain refers to the aggregate of pixels composing an image and spatial domain methods are procedures that operate directly on these pixels. Image processing function in the spatial domain may be expressed as.

$$g(x, y) = T[f(x, y)]$$

Where

$f(x, y)$ is the input image

$g(x, y)$ is the processed image and

T is the operator on f defined over some neighborhood values of (x, y) .

Frequency domain techniques are based on convolution theorem. Let $g(x, y)$ be the image formed by the convolution of an image $f(x, y)$ and linear position invariant operation $h(x, y)$ i.e.,

$$g(x, y) = h(x, y) * f(x, y)$$

Applying convolution theorem

$$G(u, v) = H(u, v) F(u, v)$$

Digital Image Processing

Where G , H and F are the Fourier transforms of g , h and f respectively. In the terminology of linear system the transform $H(u, v)$ is called the transfer function of the process. The edges in $f(x, y)$ can be boosted by using $H(u, v)$ to emphasize the high frequency components of $F(u, v)$.

Ideal Low pass Filter:

The edges and other sharp transitions (such as noise) in the gray levels of an image contribute significantly to the high-frequency content of its Fourier transform. Hence blurring (smoothing) is achieved in the frequency domain by attenuating the transform of a given image.

$$G(u, v) = H(u, v) F(u, v)$$

where $F(u, v)$ is the Fourier transform of an image to be smoothed. The problem is to select a filter transfer function $H(u, v)$ that yields $G(u, v)$ by attenuating the high-frequency components of $F(u, v)$. The inverse transform then will yield the desired smoothed image $g(x, y)$.

Ideal Filter:

A 2-D ideal lowpass filter (ILPF) is one whose transfer function satisfies the relation

$$H(u, v) = \begin{cases} 1 & \text{if } D(u, v) \leq D_0 \\ 0 & \text{if } D(u, v) > D_0 \end{cases}$$

where D is a specified nonnegative quantity, and $D(u, v)$ is the distance from point (u, v) to the origin of the frequency plane; that is,

$$D(u, v) = (u^2 + v^2)^{1/2}.$$

Figure 3 (a) shows a 3-D perspective plot of $H(u, v)$ as a function of u and v . The name ideal filter indicates that only frequencies inside a circle of radius

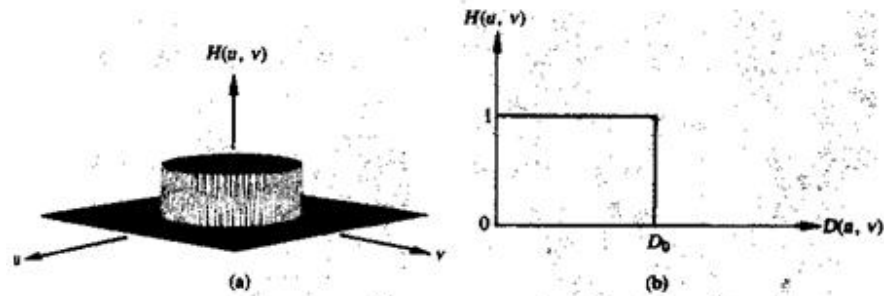


Fig.3 (a) Perspective plot of an ideal lowpass filter transfer function; (b) filter cross section.

Do are passed with no attenuation, whereas all frequencies outside this circle are completely attenuated.

The lowpass filters are radially symmetric about the origin. For this type of filter, specifying a cross section extending as a function of distance from the origin along a radial line is sufficient, as Fig. 3 (b) shows. The complete filter transfer function can then be generated by rotating the cross section 360 about the origin. Specification of radially symmetric filters centered on the $N \times N$ frequency square is based on the assumption that the origin of the Fourier transform has been centered on the square.

For an ideal lowpass filter cross section, the point of transition between $H(u, v) = 1$ and $H(u, v) = 0$ is often called the cutoff frequency. In the case of Fig.3 (b), for example, the cutoff frequency is D_0 . As the cross section is rotated about the origin, the point D_0 traces a circle giving a locus of cutoff frequencies, all of which are a distance D_0 from the origin. The cutoff frequency concept is quite useful in specifying filter characteristics. It also serves as a common base for comparing the behavior of different types of filters.

The sharp cutoff frequencies of an ideal lowpass filter cannot be realized with electronic components, although they can certainly be simulated in a computer.

Digital Image Processing

Butterworth filter:

The transfer function of the Butterworth lowpass (BLPF) of order n and with cutoff frequency locus at a distance D_0 , from the origin is defined by the relation

$$H(u, v) = \frac{1}{1 + [D(u, v)/D_0]^{2n}}$$

A perspective plot and cross section of the BLPF function are shown in figure 4.

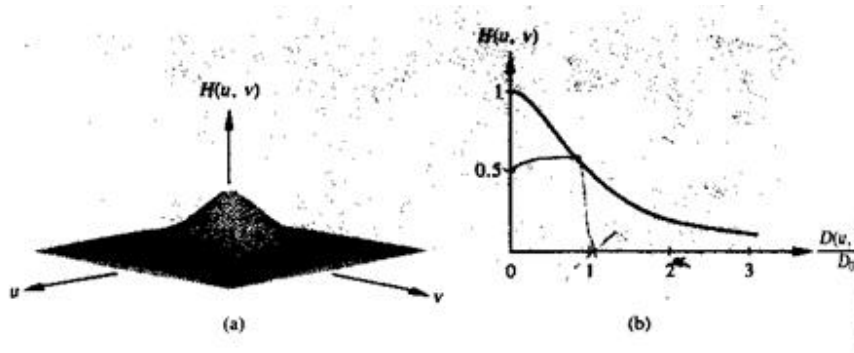


Fig.4 (a) A Butterworth lowpass filter (b) radial cross section for $n = 1$.

Unlike the ILPF, the BLPF transfer function does not have a sharp discontinuity that establishes a clear cutoff between passed and filtered frequencies. For filters with smooth transfer functions, defining a cutoff frequency locus at points for which $H(u, v)$ is down to a certain fraction of its maximum value is customary. In the case of above Eq. $H(u, v) = 0.5$ (down 50 percent from its maximum value of 1) when $D(u, v) = D_0$. Another value commonly used is $1/\sqrt{2}$ of the maximum value of $H(u, v)$. The following simple modification yields the desired value when $D(u, v) = D_0$:

$$\begin{aligned} H(u, v) &= \frac{1}{1 + [\sqrt{2} - 1][D(u, v)/D_0]^{2n}} \\ &= \frac{1}{1 + 0.414[D(u, v)/D_0]^{2n}} \end{aligned}$$

Digital Image Processing

High pass Filtering:

An image can be blurred by attenuating the high-frequency components of its Fourier transform. Because edges and other abrupt changes in gray levels are associated with high-frequency components, image sharpening can be achieved in the frequency domain by a high pass filtering process, which attenuates the low-frequency components without disturbing high-frequency information in the Fourier transform.

Ideal filter:

2-D ideal high pass filter (IHPF) is one whose transfer function satisfies the relation

$$H(u, v) = \begin{cases} 0 & \text{if } D(u, v) \leq D_0 \\ 1 & \text{if } D(u, v) > D_0 \end{cases}$$

where D_0 is the cutoff distance measured from the origin of the frequency plane. Figure 5.1 shows a perspective plot and cross section of the IHPF function. This filter is the opposite of the ideal lowpass filter, because it completely attenuates all frequencies inside a circle of radius D_0 while passing, without attenuation, all frequencies outside the circle. As in the case of the ideal lowpass filter, the IHPF is not physically realizable.

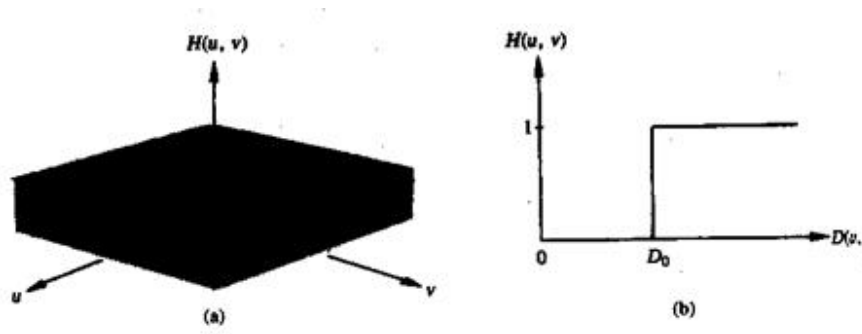


Fig.5.1 Perspective plot and radial cross section of ideal high pass filter

Butterworth filter:

The transfer function of the Butterworth high pass filter (BHPF) of order n and with cutoff frequency locus at a distance D_0 from the origin is defined by the relation

$$H(u, v) = \frac{1}{1 + [D_0/D(u, v)]^{2n}}$$

Figure 5.2 shows a perspective plot and cross section of the BHPF function. Note that when $D(u, v) = D_0$, $H(u, v)$ is down to $1/2$ of its maximum value. As in the case of the Butterworth lowpass filter, common practice is to select the cutoff frequency locus at points for which $H(u, v)$ is down to $1/\sqrt{2}$ of its maximum value.

$$\begin{aligned} H(u, v) &= \frac{1}{1 + [\sqrt{2} - 1][D_0/D(u, v)]^{2n}} \\ &= \frac{1}{1 + 0.414[D_0/D(u, v)]^{2n}} \end{aligned}$$

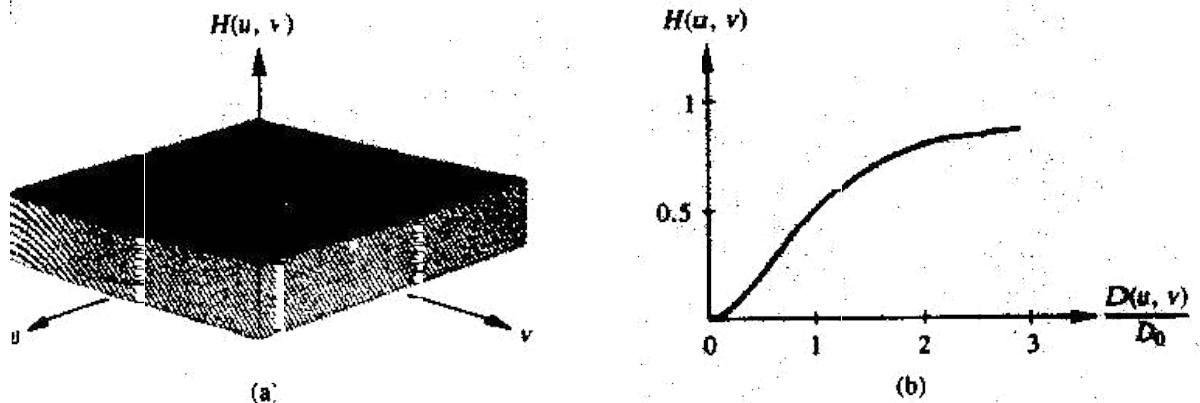


Fig.5.2 Perspective plot and radial cross section for Butterworth High Pass Filter with $n = 1$

Digital Image Processing

Gaussian Lowpass Filters:

The form of these filters in two dimensions is given by

$$H(u, v) = e^{-D^2(u, v)/2\sigma^2}$$

where, $D(u, v)$ is the distance from the origin of the Fourier transform.

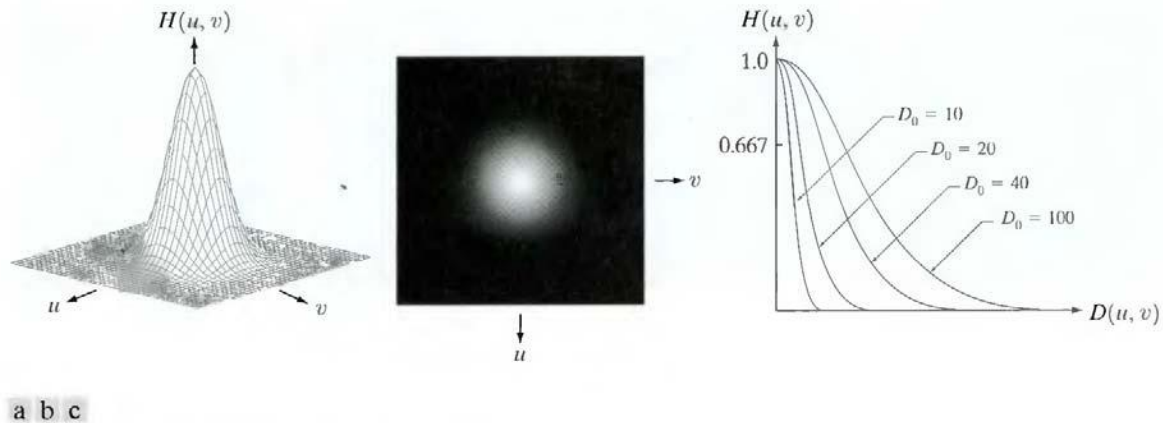


Fig.6.1 (a) Perspective plot of a GLPF transfer function, (b) Filter displayed as an image, (c) Filter radial cross sections for various values of D_0 .

σ is a measure of the spread of the Gaussian curve. By letting $\sigma = D_0$, we can express the filter in a more familiar form in terms of the notation:

$$H(u, v) = e^{-D^2(u, v)/2D_0^2}$$

where D_0 is the cutoff frequency. When $D(u, v) = D_0$, the filter is down to 0.607 of its maximum value.

Gaussian Highpass Filters:

The transfer function of the Gaussian highpass filter (GHPF) with cutoff frequency locus at a distance D_0 from the origin is given by

Digital Image Processing

$$H(u, v) = 1 - e^{-D^2(u, v)/2D_0^2}$$

The figure 6.2 shows a perspective plot, image, and cross section of the GHPF function.

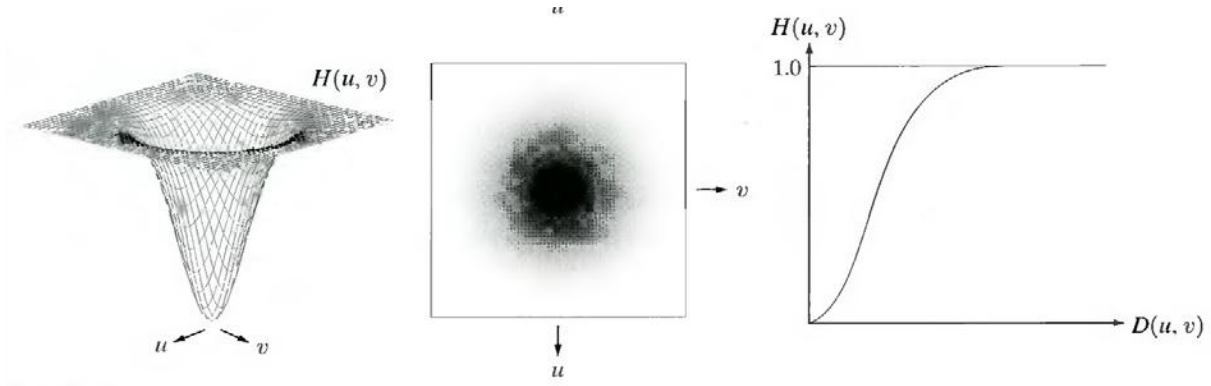


Fig.6.2. Perspective plot, image representation, and cross section of a typical Gaussian high pass filter

Even the filtering of the smaller objects and thin bars is cleaner with the Gaussian filter.

The Laplacian in the Frequency Domain:

It can be shown that

$$\mathfrak{F}\left[\frac{d^n f(x)}{dx^n}\right] = (ju)^n F(u).$$

From this simple expression, it follows that

$$\begin{aligned}\mathfrak{F}\left[\frac{\partial^2 f(x, y)}{\partial x^2} + \frac{\partial^2 f(x, y)}{\partial y^2}\right] &= (ju)^2 F(u, v) + (jv)^2 F(u, v) \\ &= -(u^2 + v^2)F(u, v).\end{aligned}$$

The expression inside the brackets on the left side of the above Eq. is recognized as the Laplacian of $f(x, y)$. Thus, we have the important result

Digital Image Processing

$$\mathfrak{F}[\nabla^2 f(x, y)] = -(u^2 + v^2)F(u, v),$$

which simply says that the Laplacian can be implemented in the frequency domain by using the filter

$$H(u, v) = -(u^2 + v^2).$$

As in all filtering operations, the assumption is that the origin of $F(u, v)$ has been centered by performing the operation $f(x, y) (-1)^{x+y}$ prior to taking the transform of the image. If f (and F) are of size $M \times N$, this operation shifts the center transform so that $(u, v) = (0, 0)$ is at point $(M/2, N/2)$ in the frequency rectangle. As before, the center of the filter function also needs to be shifted:

$$H(u, v) = -[(u - M/2)^2 + (v - N/2)^2].$$

The Laplacian-filtered image in the spatial domain is obtained by computing the inverse Fourier transform of $H(u, v) F(u, v)$:

$$\nabla^2 f(x, y) = \mathfrak{F}^{-1}\{ -[(u - M/2)^2 + (v - N/2)^2] F(u, v) \}.$$

Conversely, computing the Laplacian in the spatial domain and computing the Fourier transform of the result is equivalent to multiplying $F(u, v)$ by $H(u, v)$. We express this dual relationship in the familiar Fourier-transform-pair notation

$$\nabla^2 f(x, y) \Leftrightarrow -[(u - M/2)^2 + (v - N/2)^2] F(u, v).$$

The spatial domain Laplacian filter function obtained by taking the inverse Fourier transform of Eq. has some interesting properties, as Fig.7 shows. Figure 7(a) is a 3-D perspective plot. The function is centered at $(M/2, N/2)$, and its value at the top of the dome is zero. All other values are negative. Figure 7(b) shows $H(u, v)$ as an image, also centered. Figure 7(c) is the Laplacian in the spatial domain, obtained by multiplying by $H(u, v)$ by $(-1)^{u+v}$, taking the inverse Fourier transform, and multiplying the real part of the result by $(-1)^{x+y}$. Figure 7(d) is a zoomed section at about the origin of Fig.7(c). Figure 7(e) is a horizontal gray-level profile passing through the center of the zoomed section. Finally, Fig.7 (f) shows the mask to implement the definition of the discrete Laplacian in the spatial domain.

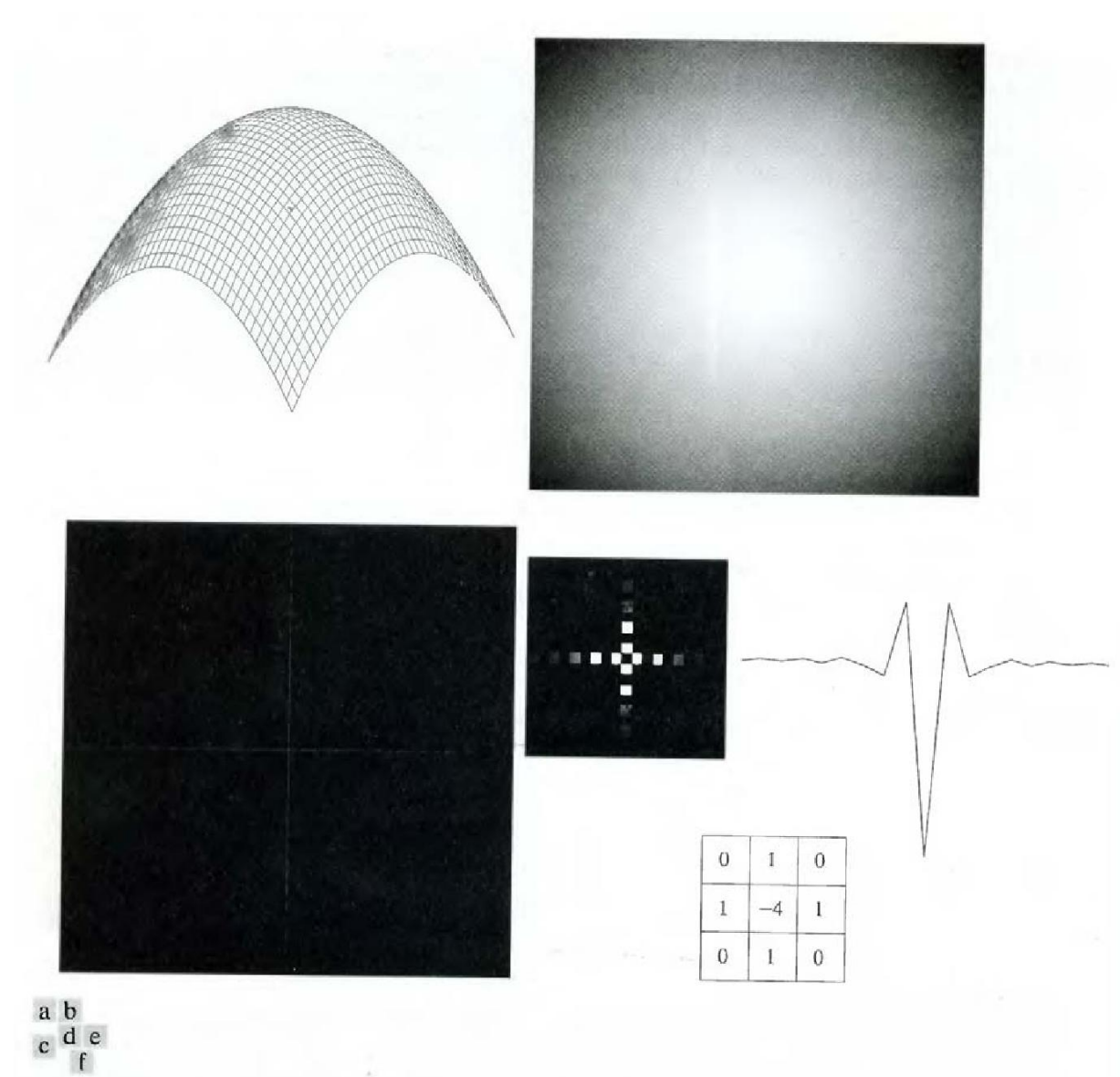


Fig.7 (a) 3 -D plot of Laplacian in the frequency domain, (b) Image representation of (a), (c) Laplacian in the spatial domain obtained from the inverse DFT of (b) (d) Zoomed section of the origin of (c). (e) Gray-level profile through the center of (d). (f) Laplacian mask

Digital Image Processing

A horizontal profile through the center of this mask has the same basic shape as the profile in Fig. 7(e) (that is, a negative value between two smaller positive values). We form an enhanced image $g(x, y)$ by subtracting the Laplacian from the original image:

$$g(x, y) = f(x, y) - \nabla^2 f(x, y).$$

High-Boost Filtering and High-Frequency Emphasis Filtering:

All the filtered images have one thing in common: Their average background intensity has been reduced to near black. This is due to the fact that the highpass filters we applied to those images eliminate the zero-frequency component of their Fourier transforms. In fact, enhancement using the Laplacian does precisely this, by adding back the entire image to the filtered result. Sometimes it is advantageous to increase the contribution made by the original image to the overall filtered result. This approach, called high-boost filtering, is a generalization of unsharp masking. Unsharp masking consists simply of generating a sharp image by subtracting from an image a blurred version of itself. Using frequency domain terminology, this means obtaining a highpass-filtered image by subtracting from the image a lowpass-filtered version of itself. That is

$$f_{hp}(x, y) = f(x, y) - f_{lp}(x, y).$$

High-boost filtering generalizes this by multiplying $f(x, y)$ by a constant $A > 1$:

$$f_{hb} = Af(x, y) - f_{lp}(x, y).$$

Thus, high-boost filtering gives us the flexibility to increase the contribution made by the image to the overall enhanced result. This equation may be written as

$$f_{hb}(x, y) = (A - 1)f(x, y) + f(x, y) - f_{lp}(x, y).$$

Then, using above Eq. we obtain

$$f_{hb}(x, y) = (A - 1)f(x, y) + f_{hp}(x, y).$$

This result is based on a highpass rather than a lowpass image. When $A = 1$, high-boost filtering reduces to regular highpass filtering. As A increases past 1, the contribution made by the image itself becomes more dominant.

Digital Image Processing

We have $F_{hp}(u, v) = F(u, v) - F_{lp}(u, v)$. But $F_{lp}(u, v) = H_{lp}(u, v)F(u, v)$, where H_{lp} is the transfer function of a lowpass filter. Therefore, unsharp masking can be implemented directly in the frequency domain by using the composite filter

$$H_{hp}(u, v) = 1 - H_{lp}(u, v).$$

Similarly, high-boost filtering can be implemented with the composite filter

$$H_{hb}(u, v) = (A - 1) + H_{hp}(u, v)$$

with $A > 1$. The process consists of multiplying this filter by the (centered) transform of the input image and then taking the inverse transform of the product. Multiplication of the real part of this result by $(-1)^{x+y}$ gives us the high-boost filtered image $f_{hb}(x, y)$ in the spatial domain.